



UNIVERSITÀ DEGLI STUDI DI MILANO
FACOLTÀ DI SCIENZE E TECNOLOGIE

DIPARTIMENTO DI FISICA
Corso di Laurea Magistrale in Fisica

Enhancing block-encoding techniques in quantum computing for algebraic speedup

Gabriele Fazio
60250A

Relatore
Prof. Enrico Prati

Correlatore
Dott. Sebastiano Corli

Anno accademico 2025/2026

“Being steadfast in the performance of your duty, act without attachment, O Arjuna, treat success and failure alike. Such equanimity is called Yoga.”

— Bhagavad Gita 2.48

ABSTRACT

Quantum computing is a field of study that has been developing over the past few decades. The main attraction of this alternative way of performing computation is the promise of a computational speedup that is otherwise unachievable through classical computers. In recent years, a new powerful quantum computing framework called block encoding (BE), aimed at performing generic matrix-vector (GEMV) multiplications, has been developed. The way BE breaks the unitary constraint of closed quantum system evolutions is through post-selecting over ancillary measurements. However, as the encoded matrix size grows, such techniques are subject to increasing failure probabilities. Ideally, being able to mitigate such a drawback would allow for the efficient computation of GEMV multiplications. It is within this context that the Thesis is set.

In Chapter 1 we study how to efficiently upload data on quantum hardware via quantum RAM (QRAM) devices, while also keeping in mind all the downsides and tradeoffs of such a powerful computational paradigm. Then, we introduce and formalize two BE techniques for generic matrices.

In Chapter 2 we develop a new class of algorithms by delineating a common way of extending both BE techniques previously shown. Such newly devised algorithms, avoiding post-selection measurements, completely bypass the failure-prone characteristic of standard BE, thus earning the name of guaranteed BE (GBE). Additionally, such new techniques also inherit the concatenating property from standard BE. So, iterative applications of such techniques result in concatenated GEMV operations. This observation plays a key role in allowing the GBEs to perform the computation of bilinear forms.

Having formally developed new algorithms, in Chapter 3 we apply numerical methods to assess the speedup by comparing two relevant tasks. First, we show that the classification task based on a single-layer neural network (NN), leveraging both BE and GBE techniques, does not provide a speedup. Contrarily, we demonstrate that in the limit of concordant signed tensors, the computation of a bilinear form exhibits an exponential speedup. We conclude our work with Chapter 4 where we summarize all the results and discuss future research directions.

TABLE OF CONTENTS

1	Introduction	1
1.1	Basic tools	2
1.2	The Quantum Ram	4
1.2.1	Efficient Quantum Random Access Memory (QRAM) imple- mentations	5
1.2.2	The binary trees	7
1.3	Block encoding techniques	9
1.3.1	Singular value decomposition	10
1.3.2	What is block encoding	12
1.3.3	The Kerenidis Prakash algorithm	14
1.3.4	The FABLE algorithm	19
1.3.5	Geometry of block encoding	23
2	Fixing the probabilistic outcomes of Block-encoding (BE) techniques	25
2.1	Extending BE techniques to failure-proof algorithms	25
2.1.1	Modifying the BE scheme	26
2.1.2	The KP extension	28
2.1.3	The Fast Approximate Block-Encodings (FABLE) extension .	30
2.1.4	Technique generalization for arbitrary signed tensors	33
2.1.5	The classical-quantum pipeline	34
2.2	Computation of bilinear forms	36
2.2.1	The concatenating property	36
2.2.2	Bilinear forms as concatenated matrix operations	40
2.2.3	The swap test bilinear form circuit	41
3	Use cases for algebraic speedups	45
3.1	Readout of generic matrix vector multiplication	45
3.1.1	A brief reminder on statistics	46
3.1.2	Precision scaling of standard BE techniques	47

3.1.3	Precision scaling of Guaranteed Block Encoding (GBE) techniques	48
3.2	The classification task	49
3.2.1	The classification task	50
3.2.2	Analysis of the singular values of a trained layers	51
3.2.3	The quantum classifier and its activation functions	58
3.2.4	Numerical analysis	59
3.2.5	Future research lines to handle the bottlenecks	64
3.3	Quantum speedup for bilinear form evaluations	67
3.3.1	Sampling relations for bilinear form circuits	68
3.3.2	Numerical analysis	69
4	Conclusions and perspectives	72
	Appendix A Software Versioning	74
	References	75

1

INTRODUCTION

During the last century a theory for describing physics at very small scales emerged, we now call it quantum mechanics. This theory marveled many generations and nowadays became widely diffused into our everyday popular culture, despite more often than not being subject to very fundamental misconceptions. This said, in spite of the very hard conceptual intricacies and very unintuitive aspects for our classical-experienced minds, quantum mechanics is formalizable via very simple postulates [1]. The quantum mechanics of closed systems, which is the formulation of interest as long as we handle non-noisy circuits, can be in fact reduced to three axioms regarding what constitutes a state, the rule that describes how states evolve in time, and the rule that tells us the probability of a measurement and the subsequent state update. A state is a vector $|\psi\rangle$ of a Hilbert space $\mathcal{H}$. Evolutions are given by unitary operators $U \in \mathcal{L}(\mathcal{H}) : \mathcal{H} \rightarrow \mathcal{H}$ (linear operators over $\mathcal{H}$ so that $UU^\dagger = \mathbb{1}$) which evolve the state via $U(t, t_0)|\psi(t_0)\rangle = |\psi(t)\rangle$. Then, measurements are given by projectors, with the Born rule telling us the probability of measuring an outcome $p(x) = \langle\psi|P_x|\psi\rangle$. The state after the measure is then the renormalized projection over the measured subspace $|\psi(x)\rangle = P_x|\psi\rangle/p(x)$.

In the second half of the past century, aside from tackling fundamental questions on quantum mechanics [2, 3] many physicists and mathematicians started realizing that such quantum systems could be exploited to perform exponentially faster calculations [4, 5]. It is within this context that just before the beginning of the new millennium some remarkable algorithms, such as the Shor factoring algorithm [6], the Grover search [7] and many others [8] were formulated. Those initial theoretical works fueled a new generation of physicists, mathematicians and computer scientists, defining a new field of research, quantum computing. In recent years, various works in the literature [9, 10] realized that many of the major

algorithms such as the Grover search [7], phase estimation [11], HHL inversion algorithm [12] and Hamiltonian simulations [13] could all be reconducted to a more general all-encompassing framework called **Quantum Singular Value Transformation (QSVT)**. The general working of **QSVT**, as the name suggests, consists in applying a polynomial function to the singular values of a certain matrix. This said, for such a complex framework to work it is mandatory to have the ability to encode non-unitary matrix evolutions onto qubits. Such a task is performed by **BE**, which is then vital for the implementation of such schemes. So, it is within this context that our work is set, and the objective to study and improve such techniques arises. We start this first pedagogical Chapter covering very briefly some fundamental notation of quantum computing in Section 1.1. Then, we immediately start covering some fundamental literature and techniques for our endeavors, starting with the **QRAM** in Section 1.2, and following with a thorough exposition on the state of the art of **BE** techniques with Section 1.3.

1.1 BASIC TOOLS

We now introduce some basic vocabulary and notions by explaining Figure 1.1. We call such a diagram a quantum circuit, which is a pictographic representation that tells us the evolution the system undergoes. The basic elements in such diagrams are the horizontal lines, where usually each corresponds to a qubit. Sometimes for graphical reasons a certain amount of lines, and so qubits, can be compacted into a single line. For example, in Figure 1.1 the last line is associated with a Hilbert space of n qubits. A qubit is a 2-dimensional quantum vector living in $\text{span}\{|0\rangle, |1\rangle\} = \mathbb{C}^2 \cong \mathcal{H}_{\text{qubit}}$, with

$$|0\rangle = (1, 0)^T \tag{1.1.1a}$$

$$|1\rangle = (0, 1)^T \tag{1.1.1b}$$

In this work we always assume that the qubits are logical, meaning they are not affected by noise but are already error-corrected. The time goes forward moving towards the right direction, and for this reason we read such diagrams from left to right. Each symbol then corresponds to some unitary applied to the respective qubits. There are many relevant gates with a name, out of all of them we briefly mention some that will be used throughout our work. In Figure 1.1 we firstly see a

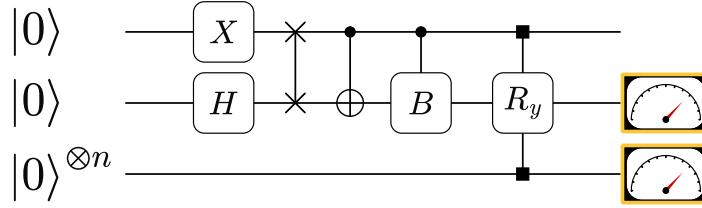


Fig. 1.1 The circuit is a pedagogical example with all diagram gates and symbols that are used in our work. Each horizontal line represents a logical qubit, which starts in the $|0\rangle$ state as indicated. The bottom line is not just a qubit but rather an aggregation of n qubits. We then apply a X (NOT) gate on the first qubit, and a Hadamard on the second. Next we swap the first two qubits. Then, we perform a CNOT, controlling the state of the first qubit and applying a NOT on the second only when the first is 1. Similarly, we can control generic gates, so we do a controlled B gate, once again checking if the first qubit is 1. The last gate is a **QRAM** call where we apply a rotation on the second qubit with an angle that depends on the value of the first and last n qubits. The circuit ends with a base measurement of all the qubits but the first.

X and H gates applied respectively to the first and second qubits. The X is called a NOT gate and is defined as

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad (1.1.2)$$

while the H is called a Hadamard gate, defined as

$$H = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} / \sqrt{2}. \quad (1.1.3)$$

Next, the following two-qubits gate, represented by the linked cross symbols and connected by a line is a SWAP, which swaps the quantum information of the two qubits, i.e. $|\psi_1\rangle|\psi_2\rangle \rightarrow |\psi_2\rangle|\psi_1\rangle$. Then, the black circle connected to the circled cross is a CNOT. A CNOT is a controlled bit flip, meaning that if the upper qubit is 1 then a X gate is applied on the second, while if the upper qubit is 0 nothing happens. We remark how the control does not actually measure the qubit, breaking the superposition, but it is a unitary operation that works on each superposed branch of the state. The same is true for also the following controlled gates. Next, we have again a black circle which is again a quantum control. The overall diagram is a quantum controlled gate, meaning that the action of the B gate is applied to the second qubit only if the value of the first qubit is 1. If the control circle was white on the inside the control would have been over the 0 value instead. The

third gate is then a **QRAM** call, where the black squares represent the qubits onto which we load a certain address, and the single qubit with the gate is the one onto which we load a certain data. We encode the data in the $|0\rangle, |1\rangle$ amplitudes via a multicontrolled R_y rotation, where R_y is defined as

$$R_y(\theta) = \begin{pmatrix} \cos(\theta/2) & -\sin(\theta/2) \\ \sin(\theta/2) & \cos(\theta/2) \end{pmatrix} \quad (1.1.4)$$

The working of the **QRAM** is discussed in far greater detail in the next Section, for now we just introduce the associated graphical notation. Then, at the end of the circuit each qubit line can end with a measurement. Unless specified otherwise the measurements are on the computational basis ($|0\rangle, |1\rangle$). We then call depth the number of parallelized gates. For example the depth of Figure 1.1 is 5, as the X and H gates can be executed in parallel since they involve different qubits. We call width the number of qubits required to run the circuit, in this case $n + 2$. We conclude this Section with a final remark on notation. Since we have a typical relation between the number of qubits n and the number of elements N they can encode which is $N = 2^n$, we denote via lower case letters the $\log_2$ of the respective uppercase quantity. In particular we use $n = \log_2 N$, $m = \log_2 M$ and $l = \log_2 L$.

1.2 THE QUANTUM RAM

We now introduce relevant literature for our work and start with the description of the **QRAM**. In the beginning of the development of quantum computing the data loading bottleneck, i.e. the cost of uploading classical data onto the quantum circuit, was a relatively neglected topic, more often than not assumed to be efficiently performable. The community [14] later grew to appreciate how handling such a task not only was not trivial, but also constituted a crucial bottleneck. It is for this reason that in the early stages, and still today, many **BE** techniques aimed to upload a matrix rely on some kind of matrix structure like sparsity or repetition of elements in given patterns. In fact, exploiting such information it is possible to greatly compress the number of operations required to query the stored data. This said, we are interested in generic purpose **Generic Matrix-Vector (GEMV)** multiplications, meaning we intend to work with no strong assumption on the matrix structure. It is within this context that the **QRAM** plays a vital role as

it allows for an efficient upload of classical data. We in fact have that without a **QRAM** the cost of uploading a matrix $A \in \mathbb{R}^{N \times M}$ would grow with $O(NM)$ which would immediately disrupt any hope of a quantum advantage for a **GEMV** multiplication task. In fact classically the cost is $O(NM)$, or $O(M)$ by leveraging a GPU. So, with Section 1.2.1 we start introducing what **QRAMs** are, how they work, and the major tradeoffs at play. Then, in Section 1.2.2 we look more closely at how to upload tensors to quantum circuits through a **QRAM** by leveraging **Binary Tree (BT)** data structures.

1.2.1 Efficient **QRAM** implementations

The literature of **QRAM** offers a great variety of alternative algorithmic implementations [15, 16]. Across these, there is a crucial tradeoff between the depth and width, i.e. we can either have shallow circuits or very small (in terms of qubits) ones. These two extremes are given by active circuit-level gate models [17] and passive hardware-level dynamic models [18]. As the first has depths that scale linearly with the number of loaded variables, i.e. $O(NM)$ for matrices, we are going to consider instead the second limit which has depths that scale logarithmically, i.e. $O(\log_2(NM))$. The big drawback of such a limit is the required number of qubits, which scales linearly, so as $O(NM)$. In spite of the presence of critics to such models [19] where errors and corrective codes could in some regimes disrupt the advantage, a core assumption for this work we make is the existence of such devices. The reason is that even though in the near term such devices could be too noisy to provide an advantage, we formulate a theoretical work that in a future where **QRAM** algorithms and implementations have been properly fine-tuned and optimized we can indeed leverage those devices to potentially provide some kind of speedup. Despite **QRAM** device implementations being still rare, just very recently a Chinese group of researcher [20] implemented a three-level qubit one (an 8-element data structure), paving the road for future more complex implementations. So, in the next paragraph we are going to start by introducing what a **QRAM** does, later focusing on the bucket brigade **QRAM** [18, 21].

A **QRAM** is a basic extension of a classical **Random Access Memory (RAM)** to a quantum setting. A **RAM** is a device that provides a query interface with an indexed data structure. In particular, given a certain address the **RAM** is able to

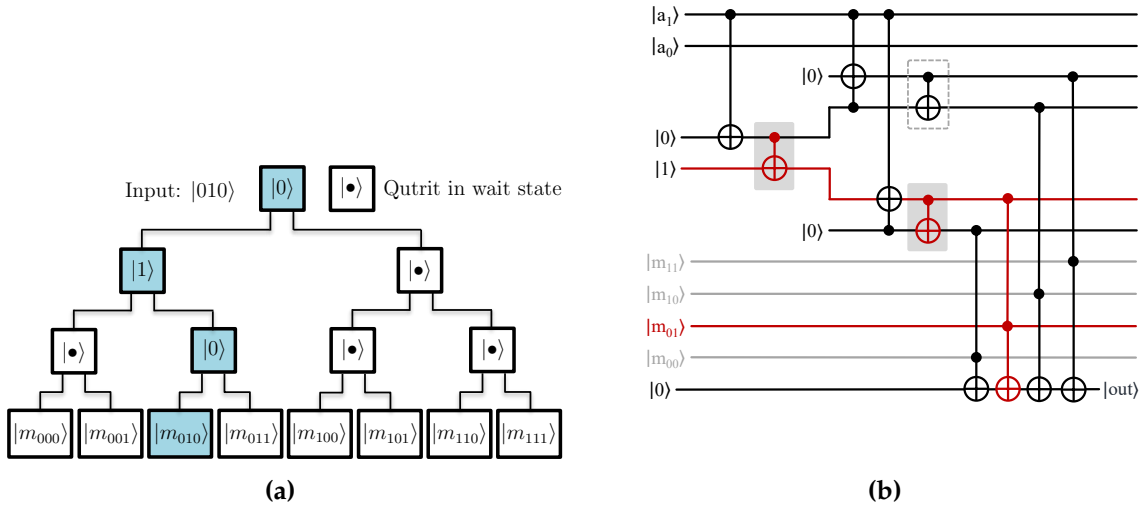


Fig. 1.2 We show the basic architecture idea for a **QRAM**. (a) In the left panel we have the logical **BT** made with qutrit switches (thus allowing a third waiting state $|\bullet\rangle$ aside from $|0\rangle$ and $|1\rangle$). All switches are in a $\bullet$ -wait state, but when traversed by the input either become 0 or 1 depending on the input. The signal thus travels the whole **BT** reaching and extracting the desired value. The signal then backtracks in order to restore the $\bullet$ -waiting state over all the switches. (b) In the right panel we offer a pragmatic implementation of a **QRAM** call. The circuit emulates such **BT** structure, entangling and moving the information from the lower 4 memory qubits up to the first two ones. The red lines delineate the path that a $|01\rangle$ input would travel. The $|a_i\rangle$ states correspond to the address register, the $|0\rangle, |1\rangle$ pairs of qubits correspond to the qutrit switches, while the $|m_{ij}\rangle$ qubits to the leaves of the **BT**, i.e. the stored data. The final $|0\rangle$ line corresponds to the logical qubit to be uploaded with the queried data.

retrieve the data stored in that particular address. So, the quantum analogue does exactly the same, but also allows for superposed queries. More formally a **QRAM** call works as follows

$$\sum_{j=1}^N \psi_j |j\rangle_{\text{address}} |0\rangle_{\text{data}} \xrightarrow{\text{QRAM}} \sum_{j=1}^N \psi_j |j\rangle_{\text{address}} |D_j\rangle_{\text{data}}, \quad (1.2.1)$$

with ψ_j the coefficient for each queried address, and D_j the data stored in the j -th address. The bucket brigade algorithm is then a possible implementation that performs Equation 1.2.1 by leveraging a **BT** of qutrit switches, where the state $|0\rangle$ means that the query addresses the left branch of the **BT**, the $|1\rangle$ the right one, while $|\bullet\rangle$ means that they stand in a waiting state for the query. Figure 1.2a shows how this algorithm works. We create a **BT** with a number of layers equal to the number of bits in the input address register, which corresponds to $\log_2 N$, i.e. the

depth of a **BT** with N leaves. The nodes that have been traversed retain the 0 or 1 value, routing the tree traversal towards the desired cell, which is then uploaded. The key advantage this algorithm brought was the idea of using waiting states for the nodes. So we do not only have 0 and 1, i.e. left and right, for the nodes that have been traversed, but also the · waiting state for all other qubits. This difference is crucial when we embed this graph traversal in a quantum setting. This single change is in fact able to reduce the number of gates from $O(N)$ to $O(\log_2^2 N)$ [18]. Also, if we perform r queries over this superposition of addresses, i.e. a number r of non-null values of ψ_j , Equation 1.2.1, we only have an entanglement across $O(r \log_2 N)$ qutrits. In Figure 1.2b an implementation of the bucket brigade for a 2-qubit address data structure. As previously mentioned we do see how we also require the four lower qubits, each prepared in a certain $|m_{ij}\rangle$ state (related to the stored information in that cell). This number, unlike the rest of the circuit, grows linearly with the number of stored entries.

1.2.2 The binary trees

We now introduce a key data structure from the **Kerenidis Prakash (KP)** [22] algorithm that allow us to efficiently upload a tensor of N components on a quantum register. The full technique is covered in greater detail in Section 1.3.3 as it is of fundamental relevance for our work. Here we just show how to leverage a **QRAM** through a **BT** data structure. So, given a tensor $\vec{x} \in \mathbb{R}^N$, with $N = 2^n$, we build a **BT** just like in Figure 1.3a. So, starting from the bottom we encode the squared values of the components in the leaves of the tree. We then sum up such values pairwise, up to the root of the tree. This way the root contains the sum of the squares of the components, i.e. the squared 2-norm

$$T_1 = \sum_{i=1}^N x_i^2 = \|\vec{x}\|_2^2. \quad (1.2.2)$$

Also, we need to store the sign information in the leaves of the tree, which is otherwise lost by squaring the value. This way when reading from it or when uploading the vector we can also correctly keep track of the sign. In Figure 1.3b we propose an indexing of such components which is useful in Section 1.3.3 for performing calculations. If we instead intend to load a matrix in the circuit, contracting it to a tensor through a **BE** technique, the process is very similar, and is shown in Figure

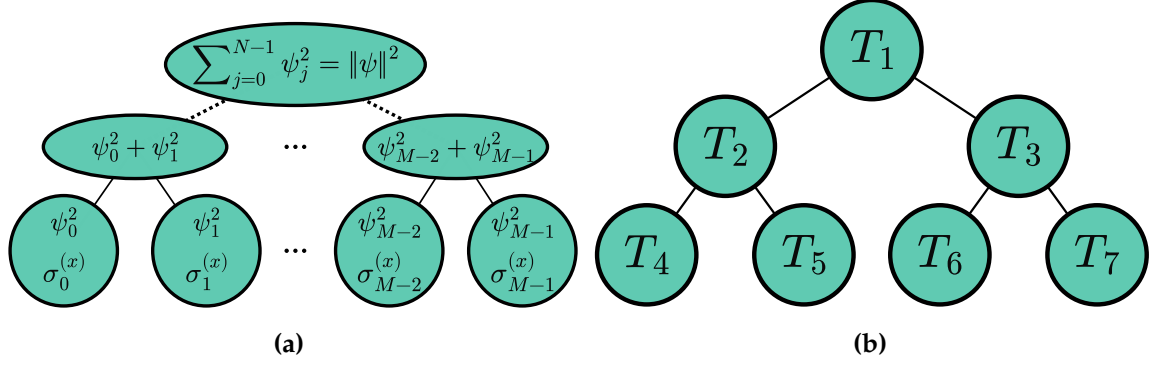


Fig. 1.3 The two panels show the **BT** structure used to upload a vector $\vec{\psi}$ onto a quantum register via a **QRAM**. (a) The left panel shows how this tree is filled with the $\vec{\psi}$ components. The last row is composed by the vector elements squared with their sign information. Then, each parent is the sum of its two children. This procedure is iterated up to the root which contains the 2-norm of the vector. (b) In the right panel we show a suitable indexing which allows for a compact notation in Section 1.3.3.

1.4. Given a matrix $A \in \mathbb{R}^{N \times M}$ we require a **BT** for each column $A_{:,i}$, i.e. the lower gray ones. On top of that we also require an overarching **BT** across those column norms, which is the greenish top one of Figure 1.4. For the modified technique we develop in Chapter 2 it is not necessary this second **BT** as it stores the values for U_L . This said, for the standard **KP BE** we require it.

We conclude this Section by briefly considering complexity costs associated with these **BT**. The main insight is that every sum performed on a given layer is completely independent, and for this reason is parallelizable. So, for example the cost of calculating the **BT** of Figure 1.3a, which is the number of sums required, is $O(N)$. If we instead leverage some parallelizing computational unit, we can parallelize all the sums at each layer, turning the overall cost to $O(\log_2 N)$. By building this **BT** structure we also automatically calculate the 2-norm, which can be found in the root. Then, if we now shift the attention to a **GEMV** multiplication $A\vec{x}$, we know this operation is made by multiplying the elements and then summing them up row by row. By leveraging the same logic we can realize that also such a task of summing up a row of M elements can be performed not just in $O(M)$ by sequentially computing the sum, but rather in $O(\log_2 M)$ by arranging the elements in the leaves of the tree (without squaring them), and then filling the tree. So, in principle the computation of $A\vec{x}$ is not either $O(NM)$ via sequential processing nor $O(M)$ parallelizing over the rows, but can even be reconducted to

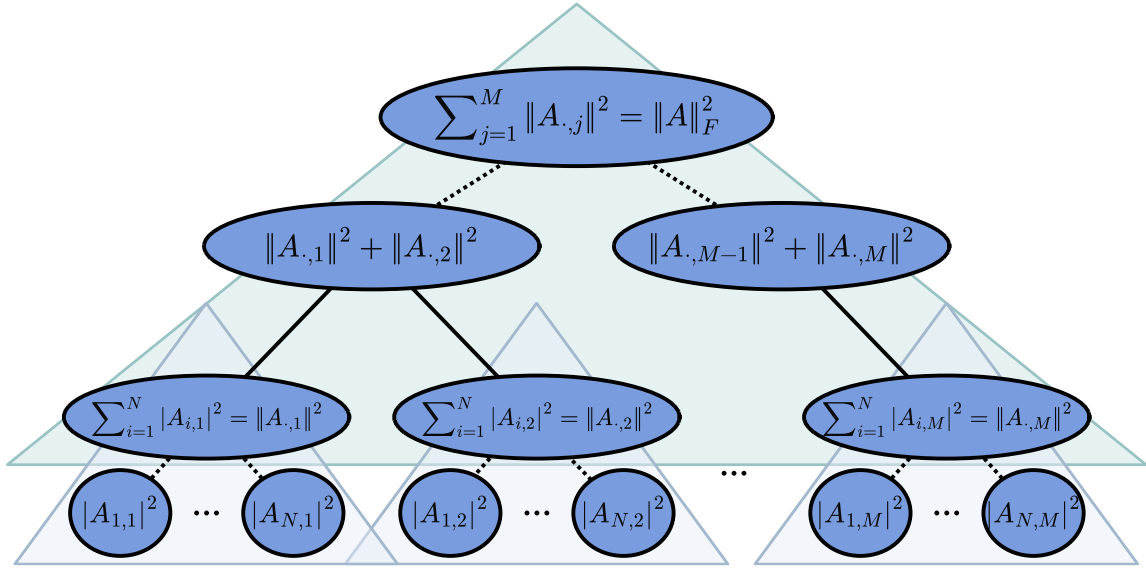


Fig. 1.4 In the panel we show the **BT** required to upload a matrix on quantum hardware via **QRAM** calls. We have two major structures that build on top of each other. For each column we have a **BT** that builds up to its column norm $\|A_{:,i}\|^2$. For graphical reasons we omit the sign information at the leaf level, but it is present just as in Figure 1.3a. We can see these **BTs** in the lower section of the panel. These are used in the calls to the **QRAM** for the U_R gate. Next, starting from the roots of those lower trees we build an overarching **BT**. This second structure is queried by the **QRAM** calls of the $U_L^\dagger$ gate. This second **BT** works just as the others, and builds itself up to the Frobenius norm $\|A\|_F^2$.

$O(\log_2 M)$. The big limitation over such great parallelization lies in the hardware capabilities. As N, M grows GPUs are not anymore able to handle all the data efficiently. Nonetheless, it is relevant to keep in mind that if N, M are small enough, such an efficient limit do exists, and is leveraged in Section 3.2.5. We conclude with a last relevant task that can be sped up through the **BT** logic, which is finding the maximum in a sequence or vector $\vec{x}$. Standard sequential implementations require $O(N)$, but exploiting the **BT** structure with all the sequence elements in the leaves, we can take the pairwise maximum of each pair. By repeating this procedure, just like in regular sports tournaments, we find at the root the maximum value of the vector. This task can thus be parallelized to only $O(\log_2 N)$.

1.3 BLOCK ENCODING TECHNIQUES

The evolution given by a closed quantum system is described by a unitary matrix U . So, given a certain state which is represented by a vector $\vec{\psi}$, the evolved state is given

by the product $U\vec{\psi}$. So, when nature evolves quantum systems it is automatically performing such matrix vector multiplications. Our objective is extending this operation to generic matrices, performing **GEMV** multiplications. In order to go towards this direction a first set of techniques to perform linear combination of unitaries (LCU) [23, 24] was developed. However, this way was not efficient to codifying generic matrices. It is within this context that **BE** [25] techniques were developed. Many **BE** techniques specialize in very specific matrix structures like sparse matrices [26–29] or more complex ones [30, 31]. It was later noticed how by leveraging the concurrent developing technology of **QRAMs** it was possible to efficiently upload dense matrices [22, 32, 33]. Since our purpose is looking for algebraic speedups in performing **GEMV** multiplications we are thus going to focus on this latter part of techniques aimed at dense matrices. This efficient upload capability is also the reason why the **QRAM** plays a vital role in allowing an efficient encoding without immediately disrupting the advantage already at the quantum upload phase. So, we start with Section 1.3.1 where we introduce the singular value decomposition, a relevant factorization method that is used throughout the exposition. Then, in Section 1.3.2 we formalize what a **BE** technique is. In Section 1.3.3 we start by introducing a first possible implementation through the **KP BE**. We immediately follow with a second technique called **FABLE** with Section 1.3.4. We then conclude with Section 1.3.5 providing some interesting observations on **BE**.

1.3.1 Singular value decomposition

We now briefly introduce what is a **Singular Value Decomposition (SVD)** and its geometric meaning. As physicists we are very familiar with diagonalization, eigenvectors and eigenvalues. This said, such concepts are well defined only for square matrices that possess certain properties. A broader diagonalization technique is thus offered by **SVD**. Given a generic matrix $A \in \mathbb{C}^{N \times M}$ it can always be factorized as

$$A = U\Sigma V^\dagger, \quad (1.3.1)$$

with U, V unitary $N \times N$ and $M \times M$ matrices, and Σ a $N \times M$ matrix with all null elements except the ones on the diagonal σ_i which we call singular values, so that σ_i are in descending order $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_{\min(N,M)}$. If the matrix was real $A \in \mathbb{R}^{N \times M}$ then U and V would be orthogonal matrices. Sticking with the more general case of A being complex, we have that the singular values are also obtainable as the

square root of the eigenvalues of $A^\dagger A$. Then, we call the columns of U the left singular vectors of A , which corresponds to the orthonormal eigenvectors of $AA^\dagger$. Similarly, the columns of V are called the right singular vectors and correspond to the eigenvectors of $A^\dagger A$.

Having introduced the formal definition we now propose a geometrical understanding of how it works. Usually when we work with eigenvectors and eigenvalues the geometrical idea is that, given a vector we look at its projections over each eigenvector. Each projected vector is then rescaled by the eigenvalue associated with that eigenvector, and then we sum all the rescaled projections forming the output vector. So, calling $|\lambda_i\rangle$ the eigenvectors of H , with the respective eigenvalues λ_i

$$H|x\rangle = \sum_i \langle \lambda_i | x \rangle \lambda_i |\lambda_i\rangle. \quad (1.3.2)$$

The idea for the **SVD** is similar. The first thing is projecting the input vector over a set of orthonormal vectors, the right singular vectors, getting numbers rather than projected vectors (just as we get $\langle \lambda_i | x \rangle$). We then rescale them by some quantity, the singular values. And finally, the big twist is that we now take those rescaled projection numbers and associate them with a new set of orthonormal vectors. This new set does not even need to live in the same space. In fact, whenever the matrix is rectangular, say $N > M$ we have that the left (output) singular vectors outnumber the right (input) ones. Since they are more it follows that some of them are always associated with a 0 factor. This observation makes sense given that it is a linear application, and it is telling us that the span of the image does not exceed that of the domain. On the contrary, whenever $N < M$ we have the opposite effect. The number of right (input) singular vectors outnumber the left (output) ones. So, we have a certain subset of input singular vectors whose components end up in the kernel of the linear application. This second case is of interest to us as in the applications of Chapter 3 we have a suitable limit when $N \ll M$. In Section 1.3.5 we explore in more depth how this observation is relevant, and what its are repercussions on the efficiency of **BE** algorithms.

1.3.2 What is block encoding

The goal of block encoding is being able to perform a non-unitary **GEMV** multiplication through the action of a closed system quantum evolution. The way we can do it is by embedding a matrix of interest A within a larger unitary, and in some way considering only that submatrix action. It is for this reason that it is called block encoding, as we are encoding A in a sub-block. The way we break the unitarity constraint is through post-selection, effectively selecting just a submatrix of the overall unitary. So, out of all the qubits a subset of those is used as ancillary measurements. Depending on the output of the measurement, if it is not successful we throw away the overall state as no clear information can be extracted. Formally, given two projectors $\tilde{\Pi}$ and Π , we call **Projected Unitary Encoding (POE)**

$$A = \tilde{\Pi} U \Pi \quad (1.3.3)$$

the encoding of A in U through the $\tilde{\Pi}, \Pi$ projectors. A special case of **POE** is **BE** where the projectors are

$$\tilde{\Pi} = \Pi = |0\rangle\langle 0|^{\otimes a} \otimes I, \quad (1.3.4)$$

which corresponds to requiring that the ancillas are all measured in the $|0\rangle$ state, while the computational register can be in whatever state. Having all the ancillas on the first space, then tensorized to the computational register, makes so that A is exactly embedded in the top left submatrix

$$U = \begin{pmatrix} A & \cdot \\ \cdot & \cdot \end{pmatrix},$$

with $\cdot$ a generic padding so that the overall matrix is unitary. Now, a subtle detail about what we said is that often we do not really encode A , but rather a rescaled version A/α with $\alpha \in \mathbb{R}^+$ a certain value called the subnormalization factor. This rescaling α comes from a fundamental property of unitary matrices. All singular values of every submatrix of a unitary are $\sigma \leq 1$. Being constrained by 1 comes from the fact that the singular values of a unitary matrix are all 1, so $\sigma_1 = \dots = \sigma_n = 1$. The least scaling factor that satisfies such a constraint is

$$\alpha_{\min} = \|A\|_2 = \sigma_{\max}^{(A)}. \quad (1.3.5)$$

In fact the 2-norm corresponds to the maximum singular value, and therefore by rescaling A with $\alpha_{\min}$ the new matrix has the top singular value to be 1 and all the others smaller, as can be understood with the **SVD** explanation of Section 1.3.1. When we design a certain **BE** we are automatically stuck with a certain α related to the algorithm. So, having a $\alpha \gtrsim \alpha_{\min}$ is the ideal limit in which a **BE** is efficiently uploading the matrix, up to the mathematical limit. What often happens is that $\alpha \gg \alpha_{\min}$. In such a limit the encoding is inefficient, which leads to a great number of failed samples, i.e. the ancillaries are not in the 0 state and so we have to discard the state. So, formally we say that given $a_L, a_R \in \mathbb{N}$ ancillary qubits, $\alpha \in \mathbb{R}^+$ the subnormalization factor, and U is a (α, a_L, a_R) **BE** of a generic matrix $A \in \mathbb{R}^{N \times M}$ when

$$\tilde{A} = (\langle 0|^{\otimes a_L} \otimes I_n) U (|0\rangle^{\otimes a_R} \otimes I_m), \quad (1.3.6)$$

with $A = \alpha \tilde{A}$. There exists then an extension of such a definition that allows for approximated encodings. This second approach is interesting as in experimental settings we always have some uncertainty of the physical gate parameters, thus requiring a definition that can account for approximated encodings, while also being relevant on the theoretical side. In fact, as we see in Section 1.3.4 there have been developed algorithms that exploit this definition to contract the depth, and thus provide a more efficient encoding. So, given $a_L, a_R \in \mathbb{N}$ ancillary qubits, $\alpha, \epsilon \in \mathbb{R}^+$ the subnormalization factor and a desired error, U is a $(\alpha, a_L, a_R, \epsilon)$ **BE** of a generic matrix $A \in \mathbb{R}^{N \times M}$

$$\|A - \alpha (\langle 0|^{\otimes a_L} \otimes I_n) U (|0\rangle^{\otimes a_R} \otimes I_m)\|_2 \leq \epsilon. \quad (1.3.7)$$

We thus use the difference between the matrix of interest and the encoded one, asking that the 2-norm of such difference be within a desired error ϵ . We conclude this Section by noting that the formal definitions have the ancillary space on the left factorized with the computational one on the right. This said, in our discussion we often ignore such rigid schemes and call **BE** even schemes where the two spaces are not in that specific ordering. So, in such cases we no longer have the A matrix neatly encoded in the top left corner, but the advantage is that we avoid the necessity of SWAPs to reroute the quantum information around the circuit. In fact, as long as we are consistent with which qubits contain the output we do not really need to reroute all computational qubits to the second register.

1.3.3 The Kerenidis Prakash algorithm

We now present what is arguably the most relevant algorithm we base our work on. Being relatively new it does not have an agreed name, but for our endeavors we call it **KP** from the two authors Kerenidis and Prakash [22]. They in fact proposed the efficient **BT** structure we briefly mentioned in Section 1.2.2 that paved the way to the later algorithm formalization into a **BE** formalism [32, 33] that we discuss. In fact the original paper predated the block encoding literature. Given a generic matrix $A \in \mathbb{R}^{N \times M}$, the technique relies on two unitaries defined via their action as

$$U_R |i\rangle |0\rangle^{\otimes n} = \frac{|i\rangle}{\|A_{\cdot, i}\|} \sum_{k=0}^{N-1} A_{k, i} |k\rangle \quad (1.3.8a)$$

$$U_L |0\rangle^{\otimes m} |j\rangle = \frac{1}{\|A\|_F} \sum_{k=0}^{M-1} \|A_{\cdot, k}\| |k\rangle |j\rangle, \quad (1.3.8b)$$

with $\|\cdot\|_F$ the Frobenius norm defined as

$$\|A\|_F = \sqrt{\sum_{i,j} A_{i,j}^2} = \sqrt{\sum_i \left(\sigma_i^{(A)}\right)^2}. \quad (1.3.9)$$

The **KP** circuit shown in Figure 1.5 is a $(\|A\|_F, n, m)$ **BE** of matrix A

$$A = \|A\|_F (\langle 0|^{\otimes n} \otimes I_m) U_L^\dagger U_R (I_n \otimes |0\rangle^{\otimes m}). \quad (1.3.10)$$

We now prove it explicitly

$$\begin{aligned} \langle 0|^{\otimes n} \langle j| U_L^\dagger U_R |i\rangle |0\rangle^{\otimes m} &= (U_L |0\rangle^{\otimes n} |j\rangle)^\dagger U_R |i\rangle |0\rangle^{\otimes m} \\ &= \left(\frac{1}{\|A\|_F} \sum_{k=0}^{M-1} \|A_{\cdot, k}\| |k\rangle |j\rangle \right)^\dagger \frac{|i\rangle}{\|A_{\cdot, i}\|} \sum_{p=0}^{N-1} A_{p, i} |p\rangle \\ &= \frac{1}{\|A\|_F} \sum_{k=0}^{M-1} \|A_{\cdot, k}\| \langle k| \langle j| \frac{|i\rangle}{\|A_{\cdot, i}\|} \sum_{p=0}^{N-1} A_{p, i} |p\rangle \\ &= \frac{1}{\|A\|_F} \sum_{k=0}^{M-1} \|A_{\cdot, k}\| \delta_{k, i} \frac{1}{\|A_{\cdot, i}\|} \sum_{p=0}^{N-1} A_{p, i} \delta_{p, j} = \frac{1}{\|A\|_F} \|A_{\cdot, i}\| \frac{1}{\|A_{\cdot, i}\|} A_{j, i} = \frac{A_{j, i}}{\|A\|_F}. \end{aligned} \quad (1.3.11)$$

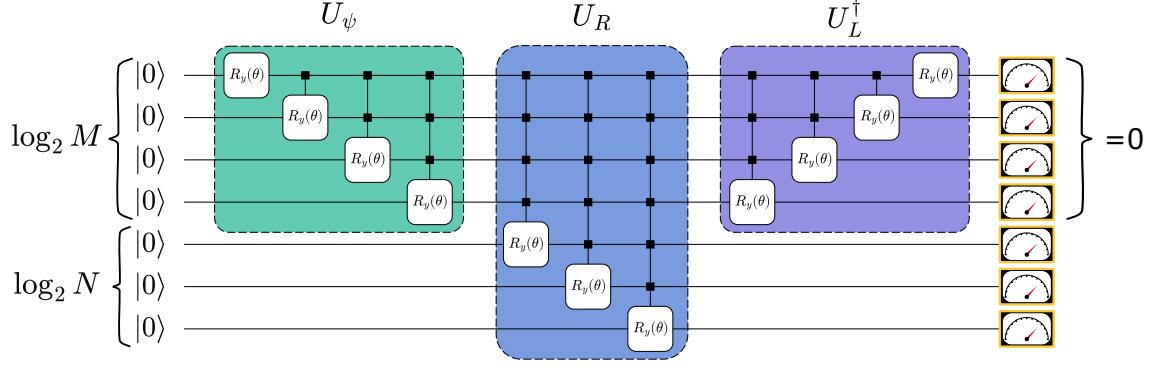


Fig. 1.5 The circuit is the implementation of the **KP** algorithm through a cascade of **QRAM** calls. The first green block serves to upload a certain input vector $\vec{\psi}$ on the quantum hardware. Next, as per **KP** we have the sequential application of U_R and $U_L^\dagger$. At the output all qubits are measured and we post-select the lower ones controlling that the top ones are 0.

So, for a generic normalized $|\psi\rangle$ input vector Equation 1.3.11 becomes

$$\langle 0|^{\otimes m} \langle j| U_L^\dagger U_R |\psi\rangle |0\rangle^{\otimes n} = \sum_{i=0}^{M-1} \frac{A_{j,i} \psi_i}{\|A\|_F}. \quad (1.3.12)$$

Since we are contracting the columns rather than the rows we already can see that we are encoding A , but we can make it even more explicit by substituting the input and outputs with projectors obtaining

$$(\langle 0|^{\otimes n} \otimes |j\rangle \langle j|) U_L^\dagger U_R (|i\rangle \langle i| \otimes |0\rangle^{\otimes m}) = \frac{A_{j,i}}{\|A\|_F} |j\rangle \langle i|. \quad (1.3.13)$$

Then, summing over all possible inputs and outputs

$$\begin{aligned} (\langle 0|^{\otimes n} \otimes I_m) U_L^\dagger U_R (I_m \otimes |0\rangle^{\otimes m}) &= \langle 0|^{\otimes n} \otimes \sum_{j=0}^{N-1} |j\rangle \langle j| U_L^\dagger U_R \sum_{i=0}^{M-1} |i\rangle \langle i| \otimes |0\rangle^{\otimes m} \\ &= \sum_{i=0}^{M-1} \sum_{j=0}^{N-1} \langle 0|^{\otimes n} \otimes |j\rangle \langle j| U_L^\dagger U_R |i\rangle \langle i| \otimes |0\rangle^{\otimes m} = \sum_{i=0}^{M-1} \sum_{j=0}^{N-1} \frac{A_{j,i}}{\|A\|_F} |j\rangle \langle i| = \frac{A}{\|A\|_F}. \end{aligned} \quad (1.3.14)$$

As anticipated we can note how in the left (output) projector the computational register is first, followed by the ancillary one, contrary to standard **BE** practice. This discrepancy could be conformed via the application of n SWAPs. Also, we note to the reader how we performed all the derivations by encoding the A ma-

trix. The circuit of Figure 1.5 in and of itself encodes the A^T matrix, and for this reason some works [33] stick with that form. This said, if we instead use the circuit to encode A^T we get A . So, as long as we properly take into account this implicit transposition within the BT structures we can avoid this confusing property.

With the first part of this Section we proved that it is possible to perform a $(\|A\|_F, n, m)$ BE via U_L and U_R . We now provide an explicit implementation for such unitaries via a successive application of QRAM calls. The circuit is still that of Figure 1.5, but this time we focus on the inside of the colored blocks. The idea is that the QRAM calls, as we defined them, can upload only a single bit of information. The way we can upload multiple bits is via a successive cascade of calls, like in Figure 1.5, which we now show it works. We can start by looking at the U_ψ gate. Here we have the fundamental pattern of QRAM calls without the additional qubit controls of U_R , which we discuss later. The first thing we have to clarify is which are the angles that are being applied by the multicontrolled R_y rotations, i.e. the QRAM calls. We rely on the binary tree of Figure 1.3a. The first R_y gate is not controlled, it is therefore a simple single qubit rotation

$$R_y |0\rangle = \left(\sqrt{\sum_{i_2, \dots, i_n \in \{0,1\}^{\times(n-1)}} \psi_{(0, i_2, \dots, i_n)_2}^2} |0\rangle + \sqrt{\sum_{i_2, \dots, i_n \in \{0,1\}^{\times(n-1)}} \psi_{(1, i_2, \dots, i_n)_2}^2} |1\rangle \right) / \|\vec{\psi}\|^2, \quad (1.3.15)$$

with $(\cdot, i_2, \dots, i_n)_2$ the binary representation of the index of the input vector $|\psi\rangle$. In simple words we are just splitting the overall 1 probability into two fractions, depending on how big the squared values with the index string that starts with 0 are with respect to the ones that start with 1. This core idea is then repeated for all the successive gates. We then continue splitting the overall probability so that at the end we arrive at the leaf level with the amplitudes being exactly the vector entries. So, formally the entries of the BT of Figure 1.3a through the indexing of Figure 1.3b are defined by the recursive relation

$$T_i = T_{2i} + T_{2i+1}. \quad (1.3.16)$$

Thus, the rotation angles are

$$\theta_i = 2 \arccos \left(\sqrt{\frac{T_{2i}}{T_i}} \right), \quad (1.3.17)$$

since

$$\begin{aligned} R_y(\theta_i) |0\rangle &= \begin{pmatrix} \sqrt{T_{2i}/T_i} & -\sin(\arccos \sqrt{T_{2i}/T_i}) \\ \sin(\arccos \sqrt{T_{2i}/T_i}) & \sqrt{T_{2i}/T_i} \end{pmatrix} \\ &= \sqrt{\frac{T_{2i}}{T_i}} |0\rangle + \sqrt{1 - \frac{T_{2i}}{T_i}} |1\rangle = \sqrt{\frac{T_{2i}}{T_i}} |0\rangle + \sqrt{\frac{T_{2i+1}}{T_i}} |1\rangle. \end{aligned} \quad (1.3.18)$$

We repeat it for all T_i entries up to the second to last layer from the bottom. In the bottom layer we in fact have to write the sign information, on top of still performing this bipartition of amplitudes. So, for the last layer the angles are

$$\theta_i = f(\sigma_i, \sigma_{i+1}) 2 \arccos \left(\sqrt{\frac{T_{2i}}{T_i}} \right) + g(\sigma_i, \sigma_{i+1}), \quad (1.3.19)$$

with σ_i, σ_{i+1} the signs associated to the T_i and T_{i+1} elements, corresponding to respective entries of the vector $\vec{x}$ to be loaded, as in Figures 1.3a and 1.3b. Those functions are

$$f(\sigma_i, \sigma_{i+1}) = \sigma_i \sigma_{i+1} \quad (1.3.20a)$$

$$g(\sigma_i, \sigma_{i+1}) = 2\pi \delta_{\sigma_i, +}, \quad (1.3.20b)$$

with $\delta_{\sigma_i, +}$ the Kronecker delta that is 1 if $\sigma_i = +1$ and 0 otherwise. The f function is then defined in order to be $-$ whenever $\sigma_i \neq \sigma_{i+1}$, and $+$ when they are equal. This way we can write the correct signs in the $|0\rangle, |1\rangle$ amplitudes. An iteration of such a procedure is

$$C R_y \sum_{p=0,1} \sqrt{\frac{T_{2i+p}}{T_i}} |0p\rangle = \sum_{p=0,1} \sqrt{\frac{T_{2i+p}}{T_i}} \left[\sqrt{\frac{T_{2(2i+p)}}{T_{2i+p}}} |0p\rangle + \sqrt{\frac{T_{2(2i+p)+1}}{T_{2i+p}}} |0p\rangle \right]. \quad (1.3.21)$$

At each step the newly added denominators T_{2i+p} always cancel with the respective prior numerators T_{2i+p} . So, at the end of all **QRAM** calls we have

$$U_\psi |0\rangle^{\otimes m} = \sum_{j=0}^{M-1} \sigma_j^{(x)} \sqrt{\frac{x_j^2}{\|\vec{x}\|_2^2}} |j\rangle, \quad (1.3.22)$$

with $\vec{x} = \|\vec{x}\| \vec{\psi}$ an unnormalized input vector, $\sigma_j^{(x)}$ the sign of component x_j . This approach holds for the upload of a vector, but the concept is very similar for U_R . The only key difference is that U_R is conditioned on the upper m qubits, where depending on the value a different column vector $\vec{A}_{\cdot,i}$ is uploaded, as per Equation 1.3.8a. The same reasoning as U_ψ holds instead for the U_L action of Equation 1.3.8b. Differently than U_R , for U_L there are in fact no additional controls, and the uploaded vector is the one formed from the column norms $\|\vec{A}_{\cdot,i}\|$. As one can see from Figure 1.5 it looks like the sequence of calls is in the opposite order. This said, the action of U_L is better understood if applied to the output, backwards in time, just as written in Equation 1.3.8b. Its action is not trivially understandable, and is best understood analytically as shown in Equation 1.3.11. This said we can think of its action as performing two things, it firstly rebalances all the different column vectors $\vec{A}_{\cdot,i}$ that are normalized by their norm, and not by a common value, and secondly it sums all those various amplitudes. The particular angles each of those **QRAM** calls use is then described in Figure 1.4, following the exact same form of Equation 1.3.17. In conclusion, the probability of measuring $|i\rangle$ in the computational register and $|0\rangle$ in the ancillary one is

$$p(|i\rangle) = \frac{|A\vec{x}|_i^2}{\|A\|_F^2 \|\vec{x}\|^2} \quad (1.3.23)$$

by relying on Equation 1.3.12. As Equation 1.3.23 clearly shows, the output is only positive, regardless of its actual sign. We conclude this Section by performing a brief analysis on the depth. The U_ψ and U_L consist of m **QRAM** calls each (considering also the first R_y rotation a **QRAM** call over a 1-element data structure). The U_R is made of n **QRAM** calls. Putting them together we have

$$O\left(2 \sum_{i=0}^{m-1} i + \sum_{i=m}^{m+n-1} i\right) = O((n+m)^2) = O(\log_2^2(NM)). \quad (1.3.24)$$

So, the depth of the **KP** scheme is $O(\log_2^2(NM))$.

1.3.4 The FABLE algorithm

A second technique that we explore is **FABLE** [26]. Unlike **KP** which heavily revolves around a **QRAM** usage, **FABLE** was ideated independently of such a device. The name itself Fast Approximate BLock Encodings is crucial in understanding the original idea. We start this Section by showing how the algorithm was originally thought, only later commenting on how **QRAM** usage can be leveraged. In Figure 1.6a we report the overall scheme, while in Figure 1.6b we show the shape of the middle oracle. The **FABLE** circuit overarching structure is given by

$$U_{\text{FABLE}} = (I_2 \otimes H^{\otimes m} \otimes I_n) \text{SWAP } O_A (I_2 \otimes H^{\otimes n} \otimes I_m), \quad (1.3.25)$$

with SWAP a set of swaps that reroute the n ancillary qubits in the lower computational register just as shown in Figure 1.6a. The oracle O_A is then defined by its action as

$$O_A |0\rangle |i\rangle |j\rangle = \left(A_{i,j} |0\rangle + \sqrt{1 - |A_{i,j}|^2} |1\rangle \right) |i\rangle |j\rangle, \quad (1.3.26)$$

with $A \in \mathbb{R}^{N \times M}$, which we can recognize to be the action of a **QRAM** call, hence its diagrammatic representation in Figure 1.6a. In order for such a definition to work we must require that $|A_{i,j}| \leq 1 \forall i, j$. We thus define a rescaling factor

$$\beta = \max_{i,j} A_{i,j}, \quad (1.3.27)$$

so that the encoded matrix A/β always optimally satisfies such a requirement. The last ingredient to be specified is then the oracle. In the original paper, instead of using a sequence of multicontrolled operations that would cost $O(NM)$ in depth (and even more if we reduce those multicontrols into elementary gates), a clever workaround is presented. By implementing the oracle as in Figure 1.6b, and leveraging the properties

$$R_y(\theta_1) R_y(\theta_2) = R_y(\theta_1 + \theta_2) \quad (1.3.28a)$$

$$X R_y(\theta) X = R_y(-\theta), \quad (1.3.28b)$$

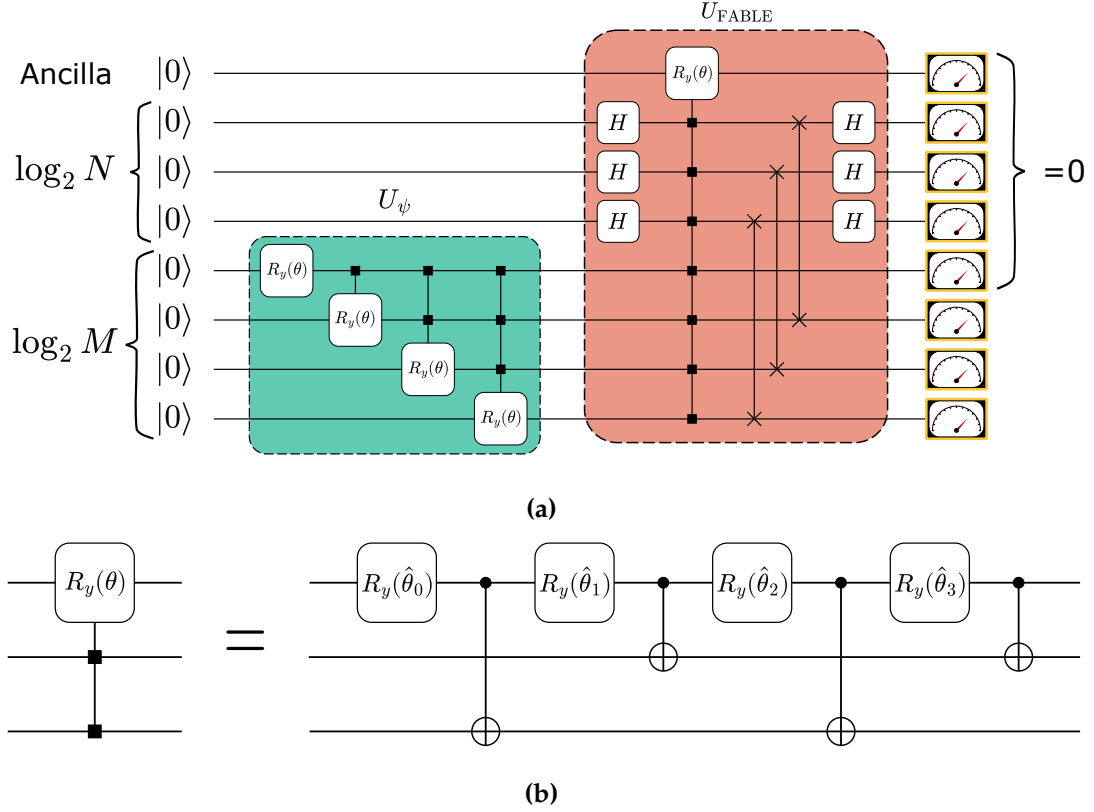


Fig. 1.6 With the two panels we present the full implementation of a **FABLE BE** application. (a) In the top panel we start by loading the input state via U_ψ through a cascade of **QRAM** calls. Next, the orange block implements the **FABLE** algorithm. We graphically represent the oracle with a **QRAM** call. At the output of the circuit we perform a base measurement over all the qubits, post-selecting the lower register whenever the top register and top ancilla are both found in the $|0\rangle$ state. (b) With the bottom panel we instead show how the original **FABLE** algorithm implemented the oracle. Through a sequential application of R_y rotations and CNOT gates disposed based on a Gray code ordering it is in fact possible to encode such an oracle with $\mathcal{O}(NM)$ elementary gates.

with X the σ_1 Pauli matrix, we have that in function of $|i\rangle$ and $|j\rangle$ the rotation is

$$\begin{aligned}
 00 : & \quad R_y(\hat{\theta}_3) R_y(\hat{\theta}_2) R_y(\hat{\theta}_1) R_y(\hat{\theta}_0) = R_y(\hat{\theta}_3 + \hat{\theta}_2 + \hat{\theta}_1 + \hat{\theta}_0), \\
 01 : & \quad R_y(\hat{\theta}_3) X R_y(\hat{\theta}_2) R_y(\hat{\theta}_1) X R_y(\hat{\theta}_0) = R_y(\hat{\theta}_3 - \hat{\theta}_2 - \hat{\theta}_1 + \hat{\theta}_0), \\
 10 : & \quad X R_y(\hat{\theta}_3) R_y(\hat{\theta}_2) X R_y(\hat{\theta}_1) R_y(\hat{\theta}_0) = R_y(-\hat{\theta}_3 - \hat{\theta}_2 + \hat{\theta}_1 + \hat{\theta}_0), \\
 11 : & \quad X R_y(\hat{\theta}_3) X R_y(\hat{\theta}_2) X R_y(\hat{\theta}_1) X R_y(\hat{\theta}_0) = R_y(-\hat{\theta}_3 + \hat{\theta}_2 - \hat{\theta}_1 + \hat{\theta}_0),
 \end{aligned} \tag{1.3.29}$$

with $\hat{\theta}_i$ the actual angles used for the rotations. These are related to the θ angles

$$\theta_{i,j} = \arccos(A_{i,j}) \quad (1.3.30)$$

by the following system of Equations

$$\begin{pmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \theta_3 \end{pmatrix} = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & -1 & 1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & 1 & -1 \end{pmatrix} \begin{pmatrix} \hat{\theta}_0 \\ \hat{\theta}_1 \\ \hat{\theta}_2 \\ \hat{\theta}_3 \end{pmatrix} = (\tilde{H} \otimes \tilde{H}) P_G \begin{pmatrix} \hat{\theta}_0 \\ \hat{\theta}_1 \\ \hat{\theta}_2 \\ \hat{\theta}_3 \end{pmatrix}, \quad (1.3.31)$$

with $\tilde{H} = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$ a scalar multiple of the Hadamard gate, and P_G is the permutation matrix that transforms binary encoding to Gray code ordering. This linear system can be efficiently solved for θ_i by a classical algorithm in $\mathcal{O}(NM \log(NM))$ via a fast Walsh-Hadamard transform [34]. So, the circuit in Figure 1.6a given by U_{FABLE} of Equation 1.3.25 is a $(1/(\sqrt{2^{n+m}}\beta), n+1, m+1)$ BE of A . We prove it by straightforward calculation acting on a certain $|j\rangle$ input state

$$\begin{aligned} |0\rangle|0\rangle^{\otimes n}|j\rangle &\xrightarrow{H^{\otimes n}} \frac{1}{\sqrt{2^n}} \sum_{k=0}^{N-1} |0\rangle|k\rangle|j\rangle \\ &\xrightarrow{O_A} \frac{1}{\sqrt{2^n}\beta} \sum_{k=0}^{N-1} \left(A_{kj}|0\rangle + \sqrt{1-|A_{kj}|^2}|1\rangle \right) |k\rangle|j\rangle \\ &\xrightarrow{\text{SWAP}} \frac{1}{\sqrt{2^n}\beta} \sum_{k=0}^{N-1} \left(A_{kj}|0\rangle + \sqrt{1-|A_{kj}|^2}|1\rangle \right) |j\rangle|k\rangle. \end{aligned} \quad (1.3.32)$$

The last layer of Hadamard gates applied to the output state gives

$$|0\rangle|0\rangle^{\otimes m}|i\rangle \xrightarrow{H^{\otimes m}} \frac{1}{\sqrt{2^m}} \sum_{l=0}^{M-1} |0\rangle|l\rangle|i\rangle. \quad (1.3.33)$$

Combining the two we get the overall action

$$\begin{aligned}
& \langle 0 | \langle 0 |^{\otimes m} \langle i | U_{\text{FABLE}} | 0 \rangle | 0 \rangle^{\otimes n} | j \rangle \\
&= \frac{1}{\sqrt{2^{n+m}} \beta} \left(\sum_{l=0}^{M-1} \langle 0 | \langle l | \langle i | \right) \left(\sum_{k=0}^{N-1} \left(A_{kj} | 0 \rangle + \sqrt{1 - |A_{kj}|^2} | 1 \rangle \right) | j \rangle | k \rangle \right) \\
&= \frac{1}{\sqrt{2^{n+m}} \beta} \sum_{k=0}^{N-1} \sum_{l=0}^{M-1} A_{k,j} \langle l | j \rangle \langle i | k \rangle = \frac{1}{\sqrt{2^{n+m}} \beta} A_{i,j}
\end{aligned} \tag{1.3.34}$$

This implementation is the basic version of the circuit. However, the depth of this circuit is still $O(NM)$, and even though the R_y rotations and CNOT gates are elementary gates so no additional complexity rolls out, the circuit is too deep to provide a speedup. It is at this point that the approximate part of the **FABLE** name comes into play. The idea is that whenever a $\hat{\theta}_i \leq \delta_c$, i.e. we are rotating the qubit by an amount below a certain desired threshold, we remove the rotation. If the matrix is sparse enough in the Walsh-Hadamard domain what happens is that many R_y are removed. This procedure is highly relevant because doing so can make various CNOT gates act sequentially. Whenever two CNOT gates acting on the same control qubit are applied they simplify to the identity. It is through this mechanism that we can effectively compress the circuit, thus exploiting such sparsity. We briefly remark how the sparsity in the Walsh-Hadamard domain is not the same as the number of 0 elements in the A matrix, but is rather a sparsity over the $\hat{\theta}$ angles obtained through the Walsh-Hadamard transform.

The one presented is the standard way **FABLE** is presented in the literature. The only notable difference we added was extending the algorithm for rectangular matrices. Also, in the next Sections like Sections 2.1.3 and 2.2.1 we neglect the SWAPs as the role they play is just to reroute the information on the lower computational register. Now, we intend to perform operations with generic matrices, and not just sparse ones. It is for this reason that instead of leveraging the oracle we discussed, which would scale as $O(NM)$ for dense matrices, we can realize that such an oracle is just a **QRAM** call. Given that for the **KP** algorithm we already assumed the existence and accessibility of such a device, we can do the same here. This way we can reduce the depth of the circuit to just $O(\log_2(NM))$, as the Hadamard gates are parallelizable, so the only scaling portion is the oracle, i.e. the single **QRAM**

call. This depth scaling is quadratically better than the **KP** option. This said, the downside of **FABLE** is that all output components allocate each a same $1/(2^{n+m}\beta)$ probability fraction, as we discuss in greater detail in Section 2.1.3. This property often leads to greater precision to resolve, which in turn means that more runs of the circuit are required. It is for this reason that we devote more attention to the **KP** algorithm. This said, also working in parallel with **FABLE** we can show how our results are not confined to just the **KP** technique, but rather they can be easily exported even to other **BE** algorithms.

1.3.5 Geometry of block encoding

We conclude this first Chapter, and Section on **BE** by providing some new intuition on the math of **BE** and some relevant bounds on the failing rate. We now introduce a quantity

$$\mathcal{L}_S = \frac{\|A\vec{x}\|^2}{\alpha^2\|\vec{x}\|^2}, \quad (1.3.35)$$

with α the subnormalization factor, and we call it logical space probability. This quantity is in fact the probability that we measure 0 on the ancillary qubits, i.e. the computation has success. The two techniques respectively have

$$\mathcal{L}_S^{(\text{KP})} = \frac{\|A\vec{x}\|^2}{\|A\|_F^2\|\vec{x}\|^2} \quad (1.3.36a)$$

$$\mathcal{L}_S^{(\text{FABLE})} = \frac{\|A\vec{x}\|^2}{2^{n+m}\beta^2\|\vec{x}\|^2}. \quad (1.3.36b)$$

For the sake of generality we can stick to Equation 1.3.35. By multiplying and dividing by $\|A\|_2^2$ we can rewrite Equation 1.3.35 as

$$\mathcal{L}_S^{(\text{KP})} = \frac{\|A\|_2^2}{\alpha^2} \frac{\|A\vec{x}\|^2}{\|A\|_2^2\|\vec{x}\|^2}. \quad (1.3.37)$$

This rewriting is really interesting as we split the dependency of the logical space probability between the goodness of the encoding via the first factor, and the captured signal with the second factor. As discussed in Section 1.3.2 the ideal **BE** technique has $\alpha = \|A\|_2$. Whenever we have $\alpha > \|A\|_2$ the first term is telling exactly the cost of such non optimality. So, the first term only depends on the encoding

algorithm, and the properties of the matrix. Next, the second term is instead purely related by how the input vector $\vec{x}$ orients itself with respect to A . Going back to the description of Section 1.3.1 on how **SVD** works, we can immediately understand that $\|A\vec{x}\|^2 = \|A\|_2^2 \|\vec{x}\|^2$ only if $\vec{x}$ is exactly along the right singular axis associated to the maximum singular value. In any other case, the projections along the other axes provide a dampening on the intensity, which makes the numerator smaller. Additionally, if $N < M$, i.e. we go down in dimension from $\vec{x} \in \mathbb{R}^M$ to $A\vec{x} \in \mathbb{R}^N$, there are projections onto the right singular axis that go into the kernel of A . This fact makes it so that on top of dimensions that are dampened, there are also others that completely vanish. So, by thinking about the norms as the output strength of the signal, with respect to the input one, we can define a quantity

$$\xi_A(\vec{x}) = \frac{\|A\vec{x}\|^2}{\|A\|_2^2 \|\vec{x}\|^2}, \quad (1.3.38)$$

that quantifies the fraction of input signal which we capture. Through this analysis we can define loose lower and upper bounds to be

$$\frac{\sigma_{\min}^2}{\alpha^2} \leq \mathcal{L}_S \leq \frac{\sigma_{\max}^2}{\alpha^2}, \quad (1.3.39)$$

with $\sigma_{\max}$ and $\sigma_{\min}$ respectively the maximum and minimum singular value of the A matrix. A final remark is that the second fraction is not intrinsically bad, but rather a vital way that **BE** needs in order to encode values within a certain scale. In fact, aside from the normalization $\|\vec{x}\|$, the scale of the output is fixed by α^2 . Whenever the algorithm needs to describe very small values the associated probability needs to be very small, thus it needs to fail many times. In the limit case in which $A\vec{x} = \vec{0}$ we in fact have that we should not expect any success shot, as that would encode some non-zero value for $\vec{p}$. So, contrary to the first term which is important to maximize, this second term is there to match the output probability to the right scale fixed by α^2 , and is not thus intrinsically bad.

2

FIXING THE PROBABILISTIC OUTCOMES OF **BE** TECHNIQUES

The previous Chapter introduced the current state of the art of **BE** techniques. As previously discussed as matrices grow, the success probability vanishes. For this reason it would be of great relevance the development of an alternative more refined technique robust against failing ancillary outcomes. With this work, I present such a scheme, able to correct the sampling failure problem. Having an always succeeding variant of standard **BE**, we thus call it **GBE**. In Section 2.1 we start by presenting the idea, later implementing it for both the **KP** and **FABLE** schemes. In Section 2.1.5 we discuss the whole pipeline of the quantum circuit, from the preprocessing to the readout, analyzing the complexity of the whole task showing that in principle a speedup is possible. Then, in Section 2.2 we show how to leverage this new technique for the task of calculating a bilinear form, either by composing successive encodings, or by using a swap test paradigm. The techniques developed in this Chapter are then used in the following one where, through a numerical analysis, we look for computational speedups in specific tasks.

2.1 EXTENDING **BE** TECHNIQUES TO FAILURE-PROOF ALGORITHMS

The purpose of this Section aims to enhance the previous **BE** schemes, in order to avoid ancillary measurements that make the algorithm incur failure when the outcome is different from 0 as discussed in Section 1.3.2. For this reason the starting point is to understand what is happening at each stage of the block encoding circuit. In Section 2.1.1 we start by analyzing the role of each component of the algorithm. Only then it will be clear that instead of doing a classically controlled state correction, rethinking the circuit is a powerful alternative which we explore. By performing a reparameterization of the tensors it is in fact possible to compute

the quantity of interest without any additional depth, achieving even a shallower circuit. In Sections 2.1.2 and 2.1.3 we discuss and formalize the extensions for KP and FABLE respectively. Since GBE algorithms have specific requirements on the signs of the tensors elements, in Section 2.1.4 we show how to overcome such a hurdle. We conclude with Section 2.1.5 by considering the whole pipeline, showing that the classical-processing phases does not constitute a bottleneck, and thus determine the overall complexity.

2.1.1 Modifying the BE scheme

We start by focusing on the KP scheme, to show afterwards that such an approach can be extended to multiple schemes such as FABLE. As discussed in Section 1.3.3 the KP scheme works through the sequential application of two unitaries U_R and $U_L^\dagger$. Reporting Equations 1.3.8a and 1.3.8b the two unitaries are defined as

$$U_R |i\rangle |0\rangle^{\otimes n} = \frac{|i\rangle}{\|A_{\cdot,i}\|} \sum_{k=0}^{N-1} A_{k,i} |k\rangle \quad (2.1.1a)$$

$$U_L |0\rangle^{\otimes m} |j\rangle = \frac{1}{\|A\|_F} \sum_{k=0}^{M-1} \|A_{\cdot,k}\| |k\rangle |j\rangle. \quad (2.1.1b)$$

For the sake of clarity we may focus on an example with $A \in \mathbb{R}^{2 \times 2}$, which we later generalize for arbitrary matrices. By feeding a quantum state through U_R we get

$$U_R \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \otimes |0\rangle = \left(\frac{A_{1,1}x_1}{\|A_{\cdot,1}\|}, \frac{A_{2,1}x_1}{\|A_{\cdot,1}\|}, \frac{A_{1,2}x_2}{\|A_{\cdot,2}\|}, \frac{A_{2,2}x_2}{\|A_{\cdot,2}\|} \right)^T. \quad (2.1.2)$$

By recalling how GEMV multiplication works, we understand the role of U_R . Its purpose is to broadcast the initial multiplication between all the elements of the matrix, with their corresponding x_i elements. The only difference is that on top of that, in order to preserve the correct norm, we also have that each element is rescaled by a certain column norm, which we remark on how it differs across the vector. In particular each x_i will be associated by a given $\|A_{\cdot,i}\|$ norm. Then, when performing a GEMV multiplication, after multiplying we add all the elements in each row. This action is performed by $U_L^\dagger$. From Equation 2.1.1b we firstly see that U_L acts only on the first register, and its action consists of mapping $|0\rangle$ to a superposition of all the columns, but rescaled by the respective column norm,

cancelling with the previous analogous rescaling as shown in Equation 1.3.11. Therefore, we have

$$U_L^\dagger U_R \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \otimes |0\rangle = \begin{pmatrix} A_{1,1}x_1 + A_{1,2}x_2 \\ A_{2,1}x_1 + A_{2,2}x_2 \\ -A_{1,1}\sqrt{\frac{\|A_{\cdot,2}\|}{\|A_{\cdot,1}\|}}x_1 + A_{1,2}\sqrt{\frac{\|A_{\cdot,1}\|}{\|A_{\cdot,2}\|}}x_2 \\ -A_{2,1}\sqrt{\frac{\|A_{\cdot,2}\|}{\|A_{\cdot,1}\|}}x_1 + A_{2,2}\sqrt{\frac{\|A_{\cdot,1}\|}{\|A_{\cdot,2}\|}}x_2 \end{pmatrix}^T / \|A\|_F. \quad (2.1.3)$$

According to how BE works we have that when the ancillary qubits are zero, i.e. the first two entries, the correct result can be sampled. In all other cases some mixed up quantity is encoded in the amplitude. We now propose a novel scheme, able to solve such an inconvenience. The main problem of BE arises from performing a sum with quantum amplitudes. In fact, whenever we sum some amplitudes through unitary means, we also end up creating unwanted operations over other parts of the quantum state (as clearly seen in Equation 2.1.3). The way standard BE handles such components is by discarding them through post-selection. The alternative we propose is instead able to retrieve information even from them. Thus, the core idea is to perform the sum in some other way. The alternative we came up with is the partial tracing of the ancillary register. We in fact have that when we measure just a subset of the qubits, i.e. the computational ones, discarding the ancillae, the probability of a given result will be given by the absolute square sum of all possible ancillary states:

$$p(I_m \otimes |j\rangle) = \sum_{k=0}^{M-1} p(|k\rangle |j\rangle). \quad (2.1.4)$$

For this reason if we were to just apply U_R as in Equation 2.1.2 and then measure the computational register only we would get

$$\begin{aligned} p(|0\rangle) &= \left| \frac{A_{1,1}x_1}{\|A_{\cdot,1}\|} \right|^2 + \left| \frac{A_{1,2}x_2}{\|A_{\cdot,2}\|} \right|^2 \\ p(|1\rangle) &= \left| \frac{A_{2,1}x_1}{\|A_{\cdot,1}\|} \right|^2 + \left| \frac{A_{2,2}x_2}{\|A_{\cdot,2}\|} \right|^2. \end{aligned} \quad (2.1.5)$$

The latter is almost the desired quantity, which can be retrieved through the following reparameterization

$$A_{i,j} \rightarrow B_{i,j} \equiv \sqrt{|A_{i,j}|}, \quad (2.1.6a)$$

$$x_i \rightarrow \psi_i \equiv \|B_{\cdot,i}\| \sqrt{|x_i|}. \quad (2.1.6b)$$

Thanks to such a rescaling, we exactly retrieve the output of $A\vec{x}$, but only as long as all the elements of A and $\vec{x}$ have a well defined sign, as the absolute value cancels them. For now let us assume that $A_{i,j}, x_k \geq 0 \ \forall i, j, k$, later in Section 2.1.4 we show how we can extend this algorithm for mixed sign tensors without negatively affecting the complexity. Then, a final note on the U_L is that given that it acts only on the ancillary register which we are tracing away, it makes no difference whether we apply it or not. Through this derivation we performed the measurements just after the U_R for the sake of clarity, as we would have ended up with the same result even with a successive application of $U_L^\dagger$. We can convince ourselves of this fact by seeing that since the action of U_L is only on the first register, it commutes with the measurement projector and thus we can cancel its action when performing the norm

$$\begin{aligned} p(|j\rangle) &= \left\| (I_m \otimes \langle j|) (U_L^\dagger \otimes I_n) U_R |\psi\rangle |0\rangle^{\otimes n} \right\|^2 \\ &= \langle \psi | \langle 0 |^{\otimes n} U_R^\dagger (U_L \otimes |j\rangle) (U_L^\dagger \otimes \langle j|) U_R |\psi\rangle |0\rangle^{\otimes n} \\ &= \langle \psi | \langle 0 |^{\otimes n} U_R^\dagger (I_m \otimes |j\rangle \langle j|) U_R |\psi\rangle |0\rangle^{\otimes n}. \end{aligned} \quad (2.1.7)$$

In conclusion, we require the BE scheme to perform at least the parallelized multiplications, while it is not required to perform the full algorithm that sums amplitudes in order to encode the desired sum in the quantum vector. The sum is thus implicitly performed by ignoring the values of the ancillary qubits, and just sampling the computational register.

2.1.2 The KP extension

In the previous Section we discussed the core idea to extend the KP algorithm, illustrating a simple case via a 2×2 matrix. We now formalize and generalize the result for arbitrary matrices. The modified circuit we obtain is shown in Figure

2.2b. We start by defining the U_ψ operation through its action on the $|0\rangle^{\otimes m} |0\rangle^{\otimes n}$ state, whose purpose is to load the normalization of $\vec{\psi}$ on the first register

$$U_\psi |0\rangle^{\otimes m} |0\rangle^{\otimes n} = |\psi\rangle |0\rangle^{\otimes n} = \sum_{i=0}^{M-1} \frac{\|B_{\cdot,i}\| \sqrt{|x_i|}}{\|\vec{\psi}\|} |i\rangle |0\rangle^{\otimes n}, \quad (2.1.8)$$

so it uploads on the upper register the reparametrized vector $\vec{\psi}$, leaving the lower register unaffected. Then, following Equation 2.1.1a U_R does

$$U_R |\psi\rangle |0\rangle^{\otimes n} = \sum_{i=0}^{M-1} \frac{\|B_{\cdot,i}\| \sqrt{|x_i|}}{\|\vec{\psi}\|} |i\rangle \frac{1}{\|B_{\cdot,i}\|} \sum_{k=0}^{N-1} B_{k,i} |k\rangle = \sum_{i=0}^{M-1} \sum_{k=0}^{N-1} \frac{B_{k,i} \sqrt{|x_i|}}{\|\vec{\psi}\|} |i\rangle |k\rangle. \quad (2.1.9)$$

Finally, the probability of measuring a given output in the computational register is

$$\begin{aligned} p(|j\rangle) &= \left\| (I_m \otimes \langle j|) U_R U_\psi |\psi\rangle |0\rangle^{\otimes n} \right\|^2 = \left\| (I_m \otimes \langle j|) \sum_{i=0}^{M-1} \sum_{k=0}^{N-1} \frac{B_{k,i} \sqrt{|x_i|}}{\|\vec{\psi}\|} |i\rangle |k\rangle \right\|^2 \\ &= \left\| \sum_{i=0}^{M-1} \frac{B_{j,i} \sqrt{|x_i|}}{\|\vec{\psi}\|} |i\rangle \right\|^2 = \sum_{i=0}^{M-1} \frac{B_{j,i}^2 |x_i|}{\|\vec{\psi}\|^2} = \sum_{i=0}^{M-1} \frac{|A_{j,i} x_i|}{\|\vec{\psi}\|^2} \end{aligned} \quad (2.1.10)$$

Therefore, as previously stated we are now able to successfully compute the desired result for tensors A and $\vec{x}$ with positive elements, without incurring into failing shots as standard BE does. A final remark on how information is stored in the output is that by having

$$\|U_R |\psi\rangle |0\rangle^{\otimes n}\|^2 = \sum_{j=0}^{N-1} \left(\sum_{i=0}^{M-1} \frac{|A_{j,i} x_i|}{\|\vec{\psi}\|^2} \right) = \sum_{j=0}^{N-1} p(|j\rangle), \quad (2.1.11)$$

we see that the vector of probabilities $\vec{p}$ is nothing more than the normalized output vector with $\|\vec{\psi}\|^2 = \|A\vec{x}\|_1$

$$\vec{p} = \frac{A\vec{x}}{\|A\vec{x}\|_1}, \quad (2.1.12)$$

with $\|\cdot\|_1$ the L_1 norm defined as

$$\|\vec{y}\|_1 \equiv \sum_{j=0}^{N-1} |y_j| \quad (2.1.13)$$

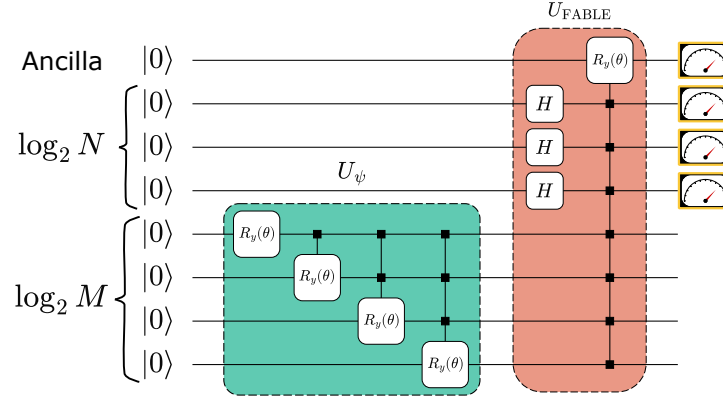


Fig. 2.1 The quantum circuit for the **GBE FABLE** scheme. The circuit is composed of a first U_ψ unitary whose purpose is to load the input vector on the lower register, and is constructed by successive **QRAM** calls, and delineated by a green rectangle. Next, in orange we have the effective **FABLE** algorithm that multiplies the input vector by the B matrix. The circuit concludes with qubit measurement over the output register and the top ancilla.

So, this algorithm encodes the given output of the multiplication (up to the normalization factor) as the fraction of shots that lands on the given $|j\rangle$ state. It then follows that to retrieve the original result we have to estimate the probability vector and multiply it by $\|A\vec{x}\|_1$. Such task can be efficiently done by recasting the operation as

$$\|A\vec{x}\|_1 = \sum_{i=0}^{N-1} \left| \sum_{j=0}^{M-1} A_{i,j} x_j \right| = \sum_{i=0}^{N-1} \sum_{j=0}^{M-1} B_{i,j}^2 |x_j| = \sum_{j=0}^{M-1} \|B_{\cdot,j}\|^2 |x_j|, \quad (2.1.14)$$

where the column norms $\|B_{\cdot,j}\|$ have already been precomputed (see Section 1.2.2). Once again, leveraging a GPU we can calculate this series in $\mathcal{O}(\log_2 M)$.

2.1.3 The **FABLE** extension

The idea for extending **FABLE** is the same as in Section 2.1.1. As shown in Section 1.3.4 the **FABLE** algorithm is

$$U_{\text{FABLE}} = (I_2 \otimes H^{\otimes m} \otimes I_n) \text{SWAP} \cdot O_B (I_2 \otimes H^{\otimes n} \otimes I_m), \quad (2.1.15)$$

with O_B the oracle that encodes a given matrix $B \in \mathbb{R}^{N \times M}$ as described in Equation 1.3.26. Just like **KP** this scheme can be decomposed into two parts, a first one that

performs the broadcasted multiplication, and a second one that sums the amplitudes creating unwanted combinations in the other components due to unitarity. We can identify those two parts with $O_B (I_2 \otimes H^{\otimes n} \otimes I_m)$ and the final layer of Hadamard gates $I_2 \otimes H^{\otimes m} \otimes I_n$. As previously discussed the role played by the SWAP is rather shallow, as its only purpose is to move the relevant state onto the computational register (lower qubits) due to the original definition of BE. Either way, as long as we are consistent with what qubit contains what information the use of SWAPs can be avoided. In Figure 2.1 we report the extended FABLE circuit. We begin the extension by looking at what is produced by the first multiplication broadcasting unitary

$$\begin{aligned} O_B (I_2 \otimes H^{\otimes n} \otimes I_m) |0\rangle |0\rangle^{\otimes n} |\psi\rangle &= \frac{1}{\sqrt{2^n}} \sum_{k=0}^{N-1} \sum_{j=0}^{M-1} \psi_j O_A |0\rangle |k\rangle |j\rangle \\ &= \frac{1}{\sqrt{2^n}} \sum_{k=0}^{N-1} \sum_{j=0}^{M-1} \left(\psi_j B_{k,j} |0\rangle + \psi_j \sqrt{1 - B_{k,j}^2} |1\rangle \right) |k\rangle |j\rangle. \end{aligned} \quad (2.1.16)$$

The main takeaway of this Equation is understanding how the multiplication is being stored and calculated in the vector. Unlike KP, which encodes a given value as a fraction of some probability of the output state (Equations 1.3.23 and 2.1.12), FABLE heavily revolves around the top ancillary qubit. The first layer of Hadamard gates puts the n ancillary qubits associated with the rows in a balanced superposition. The reason is that the algorithm does not implement a better resolution on any particular row of the output. We will see how this property will play a crucial role in differentiating FABLE from KP. Then, the role of the oracle is that of rotating the ancillary qubit in function of the stored value $|B_{k,j}| \leq 1$. This action, in union with the given probability ψ_j of having a certain $|j\rangle$, correctly encodes the amplitudes of interest $B_{k,j}\psi_j$. The extension is thus trivial. Given a desired multiplication between A and $\vec{x}$, we reparametrize them as

$$A_{i,j} \rightarrow B_{i,j} \equiv \sqrt{|A_{i,j}|}/\beta, \quad (2.1.17a)$$

$$x_i \rightarrow \psi_i \equiv \sqrt{|x_i|}, \quad (2.1.17b)$$

where, as explained in Section 1.3.4, the encoded matrix B must satisfy $|B_{i,j}| \leq 1 \forall i, j$, which is what the scaling factor β is for. Once again the assumption that $A_{i,j}, x_k \geq 0 \forall i, j, k$ is necessary, as we are not summing at the amplitude stage, but

at the probability one, and probabilities are the square of the absolute value of each multiplication, thus erasing all signs. So, just like in KP we avoid measuring the ancillary register, but this time we do continue measuring the top ancillary qubit. In fact, as we can read from Equation 2.1.16 all the coefficients $B_{k,j}\psi_j$ are stored in the amplitudes associated to $|0\rangle|k\rangle \otimes I_m$. So, we have that

$$\begin{aligned}
 p_k &= \|(\langle 0| \langle k| \otimes I_m) O_B (I_2 \otimes H^{\otimes n} \otimes I_m) |0\rangle|0\rangle^{\otimes n} |\psi\rangle\|^2 \\
 &= \left\| \frac{1}{\sqrt{2^n}} \sum_{k=0}^{N-1} \sum_{j=0}^{M-1} (\langle 0| \langle k| \otimes I_m) \left(\psi_j B_{k,j} |0\rangle + \psi_j \sqrt{1 - B_{k,j}^2} |1\rangle \right) |k\rangle |j\rangle \right\|^2 \\
 &= \left\| \frac{1}{\sqrt{2^n}} \sum_{j=0}^{M-1} B_{k,j} \psi_j |j\rangle \right\|^2 = \frac{1}{2^n} \sum_{j=0}^{M-1} |B_{k,j} \psi_j|^2 = \frac{1}{2^n \beta} \sum_{j=0}^{M-1} |A_{k,j} x_j|
 \end{aligned} \tag{2.1.18}$$

So, the way it works is by encoding a given p_k value into the probability of measuring 0 on the top ancilla while on the computational register we see $|k\rangle$. It is for this reason that the top ancilla is not really controlling whether the computation fails, rather we use it to encode the value of interest. Then, if we look at the probability of measuring $|k\rangle$ on the computational register, regardless of the top ancilla, we always get

$$\begin{aligned}
 p(|0\rangle|k\rangle) + p(|1\rangle|k\rangle) &= \frac{1}{2^n} \sum_{j=0}^{M-1} (B_{k,j} \psi_j)^2 + \frac{1}{2^n} \sum_{j=0}^{M-1} (\sqrt{1 - B_{k,j}^2} \psi_j)^2 \\
 &= \frac{1}{2^n} \sum_{j=0}^{M-1} \psi_j^2 = \frac{1}{2^n},
 \end{aligned} \tag{2.1.19}$$

where the absolute values have been simplified due to the assumptions on the signs. This is where the characteristic of giving to each output element the same resolution comes from. Equation 2.1.19 tells us in fact that the probability fraction assigned to each output component is the same. This result is not surprising as if we do not measure the top ancillary qubit we can also ignore the action of the O_B oracle, meaning that the only meaningful gates are the Hadamard gates of the first layer, whose action is exactly that which we are describing, i.e. putting in equal superposition the computational register. So, measuring the top ancillary qubit is crucial. Furthermore, it is exactly by splitting the $1/2^n$ probability into the ancilla being either 0 or 1 the way that we encode the values. So, whenever the top ancillary qubit is not 0 the computation is not failing, rather our interest is exactly

understanding the probability of 0 measurements with respect to the 1. With this observation in mind, we can conclude this extension by showing how to retrieve the original result given the newly proposed encoding. From Equation 2.1.18 it is clear we need to rescale by a factor β due to the normalization of the A matrix imposed by the constraints of FABLE, and by a factor 2^n due to the fact that the 0/1 probabilities of the N output vector components must add up to 1

$$A\vec{x} = 2^n \beta \vec{p}. \quad (2.1.20)$$

2.1.4 Technique generalization for arbitrary signed tensors

With Sections 2.1.2 and 2.1.3 we successfully extended the respective BE techniques to their always-succeeding counterparts. In the derivation we assumed that both tensors had positive entries $A_{i,j}, x_k \geq 0 \forall i, j, k$, delaying the generalization for mixed signs. We now address this issue by showing that it does not indeed constitute a problem. The core idea is that we can split the tensors into their positive and negative parts

$$A = A^+ - A^- \quad (2.1.21a)$$

$$\vec{x} = \vec{x}^+ - \vec{x}^-, \quad (2.1.21b)$$

with $A^+, \vec{x}^+$ containing only the positive elements and $A^-, \vec{x}^-$ only the negatives, i.e. $A_{i,j}^- = \max(-A_{i,j}, 0)$ and so on. Then, we can exploit linearity to decompose the initial calculation as

$$A\vec{x} = (A^+ - A^-)(\vec{x}^+ - \vec{x}^-) = A^+\vec{x}^+ + A^-\vec{x}^- - A^+\vec{x}^- - A^-\vec{x}^+. \quad (2.1.22)$$

Each of those four elements is then a GEMV multiplication of two tensors with positive elements, we in fact shifted all sign dependencies to a post-processing phase where we put together the results. Thus, the idea is to run four circuits up to a desired precision, then putting back together the results as in Equation 2.1.22. The idea of retrieving a given result from multiple circuits is in fact not something new, rather it is a well established technique present in the literature of quantum algorithms [35]. Given that only four circuits are required, independently from the size of the tensors, the class of complexity is unaltered. Furthermore, on the technological side it is also not a problem as we are not required to quadruple the

number of qubits that are going to interact and form entangled states, rather we are only required to have four independent circuits run. These executions can be done both sequentially, so after performing the first set of runs on the first circuit the second one is loaded on the same hardware and samples are made, up to the third and fourth, or also in parallel, meaning that at the same time we run on separated qubits all four circuits sampling them all at the same time.

2.1.5 *The classical-quantum pipeline*

In the previous Sections we proposed modifications of BE techniques that overcome the failing characteristic typical of BE. In this Section we delineate the overall working pipeline which is reported in Figure 2.2, pointing out the crucial interplay of classical and quantum hardware, and how both are necessary for a proper speedup.

The setup is the following: we are given a certain $A \in \mathbb{R}^{N \times M}$ matrix beforehand, and we are asked to be able to perform GEMV multiplication of that matrix on a given set of vectors that will be provided to us on the fly. Thus, the pipeline is mainly divided into four stages: an offline preprocessing phase, an online preprocessing phase, the execution of the quantum circuit and the postprocessing of the sampled data to retrieve the result. The first offline preprocessing is performed on the matrix, and it revolves around calculating the BTs of the modified matrix (see Equations 2.1.6a or 2.1.17a) in order to upload the matrix via a QRAM as described in Section 1.2.1. This classical operation takes $\mathcal{O}(NM)$, which does not constitute a classical bottleneck, and can even be parallelized. However, as it happens in the offline preprocessing and is just a one-time operation, its complexity is not taken into account for the online complexity. Having completed the offline preparation, we then start the online processing of data. A given vector $\vec{x}$ is provided, and in order to correctly upload it on the quantum circuit it is necessary to firstly reparametrize it as described in Equation 2.1.6b or 2.1.17b, and then build the BT for the QRAM calls. The normalization is then automatically taken care of by the QRAM, with the norm being stored in the root of the BT. The cost of the reparametrization is $\mathcal{O}(M)$, parallelizable via a GPU to $\mathcal{O}(1)$, while the cost of building the BT is $\mathcal{O}(M)$, but parallelizable only down to $\mathcal{O}(\log_2 M)$ as described in Section 1.2.2. Having classically processed the matrix and the vector, and set up the QRAM for a quantum upload, it is now time to run the quantum circuit. As

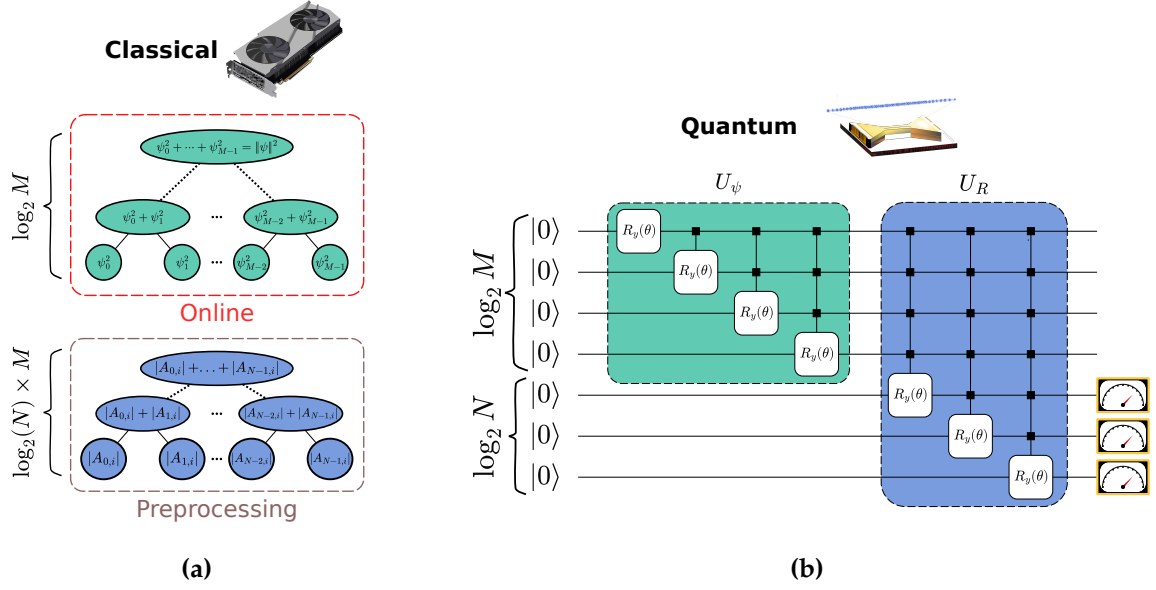


Fig. 2.2 Pipeline for matrix vector multiplication. (a) To store all the coefficients of the matrix A , from $|A_{j,i}|$ up to their row norm $\|A_{:,i}\| = \sum_j |A_{j,i}|$, an amount of M binary trees are required. While such matrix weights can be uploaded in preprocessing, the inputs can be processed in the binary tree via a classical parallel processor. (b) Once the BT stores all the information, the data can be encoded in the circuit and processed by the QRAM and the QPU. The KP GBE circuit is composed of a first green block whose purpose is to load the input vector, followed by the actual KP unitary which multiplies an input vector with the B matrix. The circuits end with qubit measurement over the output register.

shown in Section 1.3.3 the circuit depth of a standard KP BE (just like the depth of the proposed modified counterpart) is $O(\log_2^2(NM))$, while for both FABLE approaches we have $O(\log_2(NM))$. This circuit is then sampled a certain number of times. As N increases we will have more components to sample, which in turn means that the encoded values will be smaller (as the probabilities are bound to sum to 1), and therefore a higher precision will be required. However, there is not a pre-defined scaling for the number of runs with respect to either N or M as the required precision depends on the task of interest. This last point will be expanded on and discussed at length in the next Chapter. Then, assuming we performed this pipeline for up to four distinct tensors combinations $A^+x^+, A^-\vec{x}^-, A^+\vec{x}^-, A^-\vec{x}^+$, we classically collect the various sampling probabilities and rescale them back as shown in Equations 2.1.12 and 2.1.20. Then, as discussed in Section 2.1.4 we put together the various outputs of the circuits via Equation 2.1.22. These two final postprocessing tasks require respectively $O(N)$ and $O(1)$, with the former parallelizable to $O(1)$ too. The pipeline is summarized in Figure 2.2.

2.2 COMPUTATION OF BILINEAR FORMS

In Section 2.1 we formalized a way to promote two standard BE techniques to a failure-proof counterpart. A key feature of BE techniques is that they can be applied sequentially, and their effect is that of performing multiple sequential GEMV multiplications. However, it also comes with the downside that the number of ancillary qubits that have to be 0 increases, meaning that the fraction of successful shots is greatly reduced. A particular use case we have for a double GEMV multiplication is that of calculating a bilinear form $\vec{y}^T A \vec{x}$ where the last vector $\vec{y}$ can be thought of as a single column matrix. This task is highly relevant as it is a core routine for algorithms like Support Vector Machine (SVM) or quantum many-body simulations [36–38]. So, in Section 2.2.1 we are going to firstly inquire whether the newly proposed modified techniques do inherit the concatenating property, and if so under which conditions. Then, in Section 2.2.2 we look at a relevant case where we implement a bilinear form, finding that it is indeed a very suitable setting in which the unfavorable conditions are mitigated and it is possible to find a regime with a speedup with respect to classical computation. We then conclude with Section 2.2.3 where we present an alternative approach to calculate bilinear forms leveraging the swap test circuit structure.

2.2.1 The concatenating property

We are interested in the computation of a bilinear form $\vec{y}^T A \vec{x}$. We can in fact think of bilinear forms as a double GEMV, with $\vec{y}^T$ the second matrix applied on the first output vector $A \vec{x}$. It is for this purpose that we start by reviewing how standard BE algorithms concatenate. We are going to consider and perform calculations with KP, but the same can be repeated for FABLE too. Given two matrices $A \in \mathbb{R}^{N \times M}$ and $B \in \mathbb{R}^{L \times N}$, we build a circuit that encodes $B \cdot A$. In order to do this operation we can build a circuit as in Figure 2.3a. We thus have an initial register of $m = \log_2 M$ qubits where we load the state on, and a computational register where the output of the first GEMV multiplication with the A matrix goes, made of $n = \log_2 N$ qubits. Then, the idea is to take this output register and use it as the input of the following matrix. So, we are going to reuse the n qubits, and on top of that we require a new computational register of $l = \log_2 L$ qubits, onto which the final result is written on. Then, in order for the algorithm to have success, all $n + m$ ancillary qubits at the end

have to be measured to be 0, otherwise the computation fails and the computational register does not contain the desired output. Formally, using Equation 1.3.11

$$\begin{aligned}
& \langle 0|^{\otimes m} \langle 0|^{\otimes n} \langle j|_l U_L^\dagger(B) U_R(B) U_L^\dagger(A) U_R(A) |i\rangle_m |0\rangle^{\otimes n} |0\rangle^{\otimes l} \\
&= \langle 0|^{\otimes n} \langle j|_l U_L^\dagger(B) U_R(B) \left((\langle 0|^{\otimes m} \otimes I_n) U_L^\dagger(A) U_R(A) |i\rangle_m |0\rangle^{\otimes n} \right) |0\rangle^{\otimes l} \\
&= \langle 0|^{\otimes n} \langle j|_l U_L^\dagger(B) U_R(B) \left(\sum_{k=0}^{N-1} (\langle 0|^{\otimes m} \otimes |k\rangle \langle k|) U_L^\dagger(A) U_R(A) |i\rangle_m |0\rangle^{\otimes n} \right) |0\rangle^{\otimes l} \\
&= \langle 0|^{\otimes n} \langle j|_l U_L^\dagger(B) U_R(B) \left(\sum_{k=0}^{N-1} \frac{A_{k,i}}{\|A\|_F} |k\rangle \right) |0\rangle^{\otimes l} \\
&= \sum_{k=0}^{N-1} \frac{A_{k,i}}{\|A\|_F} \langle 0|^{\otimes n} \langle j|_l U_L^\dagger(B) U_R(B) |k\rangle |0\rangle^{\otimes l} = \sum_{k=0}^{N-1} \frac{B_{j,k} A_{k,i}}{\|B\|_F \|A\|_F}.
\end{aligned} \tag{2.2.1}$$

This concatenation is easily generalizable for an arbitrary number of matrices, even though the more matrices the bigger will be the normalization factor given by the product of the Frobenius norms. We now repeat this calculations for the new extended algorithms as shown in Figure 2.3b, in order to look where possible problems can arise. Assuming $A_{i,j}, B_{k,l}, x_m \geq 0$ (generalizable to generic sign tensors via multiple circuits just as explained in Section 2.1.4) and using Equation 2.1.9

$$\begin{aligned}
U_R(B) U_R(A) |\psi\rangle_m |0\rangle^{\otimes n} |0\rangle^{\otimes l} &= U_R(B) \sum_{i=0}^{M-1} \sum_{k=0}^{N-1} \frac{\sqrt{A_{k,i}} \sqrt{x_i}}{\|\vec{\psi}\|} |i\rangle_m |k\rangle_n |0\rangle^{\otimes l} \\
&= \sum_{i=0}^{M-1} \sum_{k=0}^{N-1} \frac{\sqrt{A_{k,i}} \sqrt{x_i}}{\|\vec{\psi}\|} |i\rangle_m U_R(B) |k\rangle_n |0\rangle^{\otimes l} \\
&= \sum_{i=0}^{M-1} \sum_{k=0}^{N-1} \frac{\sqrt{A_{k,i}} \sqrt{x_i}}{\|\vec{\psi}\|} |i\rangle_m \sum_{j=0}^{L-1} \frac{\sqrt{B_{j,k}}}{\|B_{\cdot,k}\|} |k\rangle_n |j\rangle_l.
\end{aligned} \tag{2.2.2}$$

The main problem of this Equation is that, unlike $\|A_{\cdot,i}\|$ which is manually compensated by the contracted vector $\vec{\psi}$ as by Equation 2.1.6b, the $\|B_{\cdot,k}\|$ are not corrected by any other object. In order for the cancellation to happen we have to rely on a tensor which is indexed by k , so we cannot push once again the task onto the $\vec{x}$ vector. Similarly we cannot make B itself do it as changing the row by a normalization factor would get canceled by the updated normalization itself. Therefore, just as

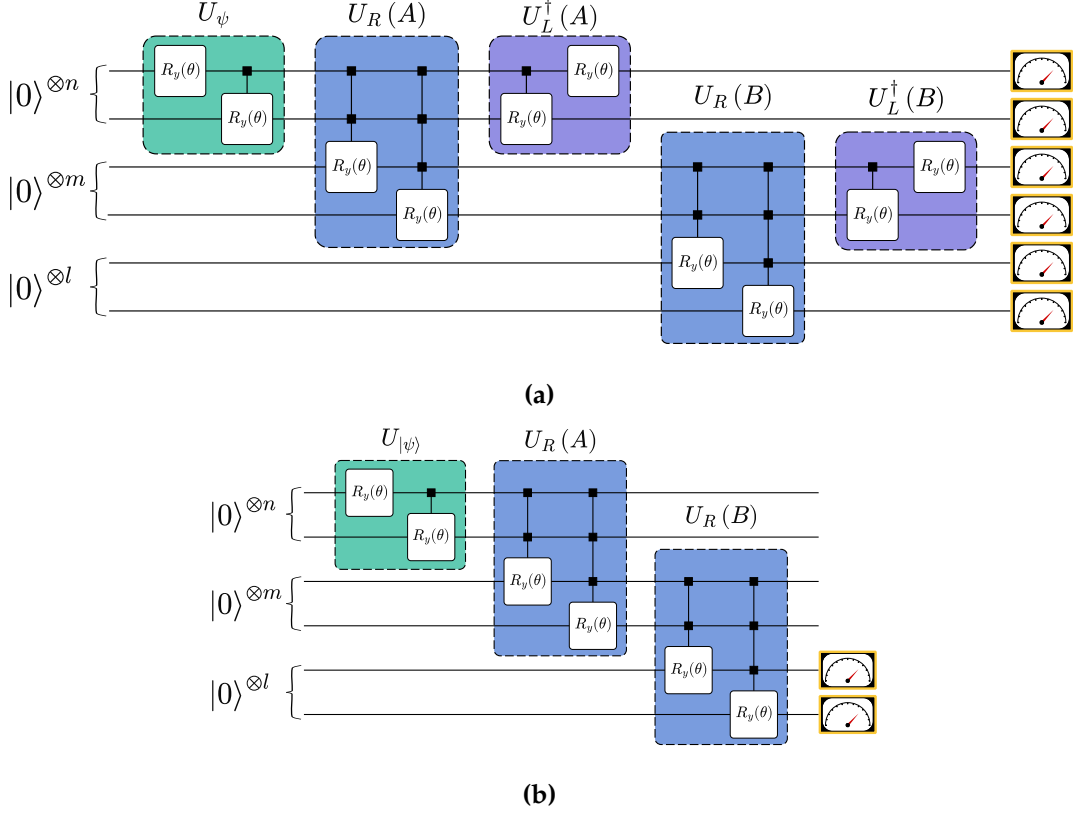


Fig. 2.3 Both panels show the circuit implementation of concatenated **GEMV** multiplications. (a) The top panel is the concatenation of two standard **BE**, following the green block which uploads the input vector. As per standard **BE**, the circuit ends with qubit measurements over all qubits, where the last l will be subject to a post-selection based on the 0 outcome of all the $n + m$ previous ones. (b) The bottom panel shows the modified counterpart. We no longer require the $U_L^\dagger$ unitaries, and the measurements are performed only on the computational register, i.e. the lower qubits.

before the only tensor able to rescale properly the quantity is the one contracted to B , i.e. A . For this reason we have to perform the rescaling

$$A_{k,i} \rightarrow A'_{k,i} \equiv A_{k,i} \|B_{\cdot,k}\|^2. \quad (2.2.3)$$

The complexity of this operation is $\mathcal{O}(NM)$, which can be parallelized. In contrast, if we were to perform the matrix multiplication in a preprocessing phase, the matrix-vector multiplication would take $\mathcal{O}(LMN)$, parallelizable in various ways depending on which dimensions are smaller. The benefit is that the preprocessing cost is smaller than performing the actual multiplication, but the downside is that in order to do the operation on a quantum circuit with the **GBE** approach we cannot

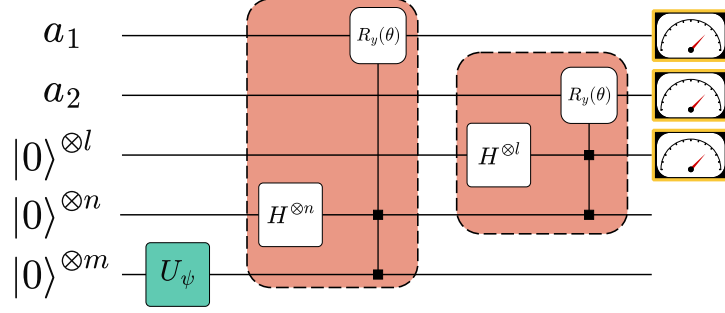


Fig. 2.4 The concatenation of two **FABLE GBE** circuits. After the upload of the input vector through the first green block, we perform a first **FABLE GBE** as usual, then using the first output register as the input for the next **FABLE** application. The circuit ends with qubit measurements over both top ancillae, and over the output register of l qubits.

just stack the gates for whatever combination of matrices, but every matrix has to be modified in order to correct the following one. Notably, this correction can be avoided whenever all norms $\|B_{\cdot,k}\|$ are 1. This observation will play a crucial role in the next Section, as it allows for a very interesting application.

We have successfully shown that in order to concatenate **KP GBEs** we need to fine-tune the quantum embeddings of the matrices at play. We in fact have to account for the very intricate interplay between $U_L^\dagger$ and U_R . In the standard technique it is the action of $U_L^\dagger$ that rebalances all the various elements rescaled by different norms. Since the extended technique is unaffected by $U_L^\dagger$ we end up having to manually handle it, thus justifying the need for such fine-tuning. However, as previously shown **FABLE** is logically much more straightforward. This property is also reflected by Equation 2.1.17b as $\vec{x}$ does not have to be rescaled by some norm. It is for this reason that **FABLE** lends itself nicely for concatenated applications. Using Equation 2.1.16, the action of the circuit in Figure 2.4 is

$$\begin{aligned}
 & O_B \left(I_4 \otimes H^{\otimes l} \otimes I_{n+m} \right) O_A \left(I_{4+l} \otimes H^{\otimes n} \otimes I_m \right) |0\rangle_{a1} |0\rangle_{a2} |0\rangle^{\otimes l} |0\rangle^{\otimes n} |j\rangle_m \\
 &= O_B \left(I_4 \otimes H^{\otimes l} \otimes I_{n+m} \right) \sum_{k=0}^{N-1} \frac{\sqrt{A_{k,j}} |0\rangle_{a1} + \sqrt{1-A_{k,j}} |1\rangle_{a1}}{\sqrt{2^n \beta_A}} |0\rangle_{a2} |0\rangle^{\otimes l} |k\rangle_n |j\rangle_m \\
 &= \sum_{k=0}^{N-1} \frac{\sqrt{A_{k,j}} |0\rangle_{a1} + \sqrt{1-A_{k,j}} |1\rangle_{a1}}{\sqrt{2^n \beta_A}} \left(\sum_{i=0}^{L-1} \frac{\sqrt{B_{i,k}} |0\rangle_{a2} + \sqrt{1-B_{i,k}} |1\rangle_{a2}}{\sqrt{2^l \beta_B}} |i\rangle_l \right) |k\rangle_n |j\rangle_m,
 \end{aligned} \tag{2.2.4}$$

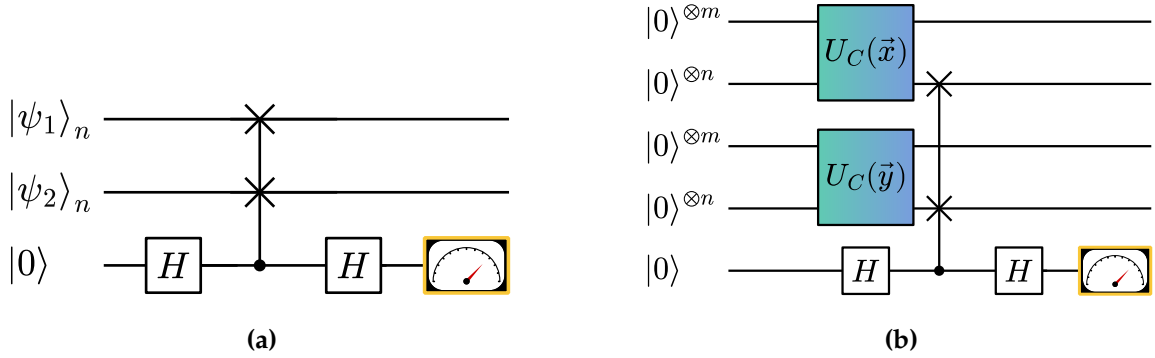


Fig. 2.5 (a) The standard swap test, where through the use of ancillary qubits it is possible to compute the inner product of two states loaded in the respective registers. (b) We leverage the swap test circuit structure to perform the inner product of the two vectors, which ultimately encodes the bilinear form of interest. The two green-blue blocks are composed of a (green) upload of a certain $\vec{x}$ or $\vec{y}$ vector (suitably reparametrized), followed by the (blue) multiplication of the C matrix through a **KP GBE** scheme, as described in Equations 2.2.12a and 2.2.12b. Then, differently from a standard swap test, the controlled SWAP is performed only on the respective two output registers of the **KP** blocks, ignoring the respective ancillary registers. The circuit, just as the swap test, concludes with a qubit measurement over the ancilla.

for which p_i is encoded in the probability of measuring i on the final computational register, while also measuring 0 on both ancillas

$$p_i = \left\| \frac{1}{\sqrt{2^{n+l}\beta_A\beta_B}} \sum_{k=0}^{N-1} \sqrt{B_{i,k}A_{k,j}} |k\rangle_n \right\|^2 = \frac{1}{2^{n+l}\beta_A\beta_B} \sum_{k=0}^{N-1} B_{i,k}A_{k,j}. \quad (2.2.5)$$

The latter Equation correctly proves that no additional modification on the matrices is required, but the downside is that we introduced some degree of failing into the algorithm. We in fact have that both ancillas have to be 0 in order for the computation to be successful. However, the failing rate is very moderate as it is independent of the size of the matrices, but rather only depends on the number of concatenated operations. The number of top ancillas is in fact equal to the number of subsequent multiplied matrices.

2.2.2 Bilinear forms as concatenated matrix operations

In the previous Section we showed that it is indeed possible to concatenate not only standard **BE**, but even the **GBEs**. We now focus on a special case, the computation

of bilinear forms. Given a matrix $A \in \mathbb{R}^{N \times M}$, and two vectors $\vec{x} \in \mathbb{R}^M$ and $\vec{y} \in \mathbb{R}^N$, a bilinear form is the mapping that the A matrix does on the two vectors into a number via $\vec{y}^T A \vec{x}$. The core idea for the **KP GBE** is treating the bilinear form as a sequence of two multiplied operations, where we encode the $\vec{y}^T$ vector into the following matrix

$$Y = \begin{pmatrix} \tilde{y}_1 & \cdots & \tilde{y}_N \\ \sqrt{1 - \tilde{y}_1^2} & \cdots & \sqrt{1 - \tilde{y}_N^2} \end{pmatrix}, \quad (2.2.6)$$

with $\tilde{y}_i = \sqrt{y_i} / (\max_j \sqrt{y_j})$ in order for the square roots to be well defined. We call $\sqrt{y_{\max}} = \max_j \sqrt{y_j}$. Finding the maximum is an operation that takes $\mathcal{O}(N)$, parallelizable with a GPU down to $\mathcal{O}(\log_2 N)$ via a **BT** lookup algorithm. As anticipated, the remarkable property of this matrix is that all column norms are exactly 1 by construction, and for this reason no additional changes to the A embedding are required. Therefore, as shown in Figure 2.3b and by Equation 2.2.2

$$U_R(Y) U_R(A) |\psi\rangle_m |0\rangle^{\otimes n} |0\rangle = \sum_{i=0}^{M-1} \sum_{k=0}^{N-1} |i\rangle_m |k\rangle_n \left[\frac{\sqrt{y_k} A_{k,i} x_i}{\|\vec{\psi}\| \sqrt{y_{\max}}} |0\rangle + \frac{\sqrt{(1 - y_k^2)} A_{k,i} x_i}{\|\vec{\psi}\| \sqrt{y_{\max}}} |1\rangle \right] \quad (2.2.7)$$

working in a similar fashion to the **FABLE** technique as the output value is encoded into the probability of measuring 0 on a given singular ancillary qubit.

2.2.3 The swap test bilinear form circuit

We conclude this Chapter by displaying a final technique, which differs from the prior in structure, yet performs nonetheless the computation of a bilinear form. The idea is to leverage the swap test algorithm structure, and adapt it to our use case. For this reason we start by looking at the standard swap test routine [39], on why

and how it works. The circuit is shown in Figure 2.5a, and works as follows

$$\begin{aligned}
 (I_{2n} \otimes H) \text{CSWAP} (I_{2n} \otimes H) |\psi_1\rangle_n |\psi_2\rangle_n |0\rangle &= (I_{2n} \otimes H) \text{CSWAP} |\psi_1\rangle |\psi_2\rangle \frac{|0\rangle + |1\rangle}{\sqrt{2}} \\
 &= (I_{2n} \otimes H) \frac{|\psi_1\rangle |\psi_2\rangle |0\rangle + |\psi_2\rangle |\psi_1\rangle |1\rangle}{\sqrt{2}} \\
 &= \frac{1}{2} \left[(|\psi_1\rangle |\psi_2\rangle + |\psi_2\rangle |\psi_1\rangle) |0\rangle + (|\psi_1\rangle |\psi_2\rangle - |\psi_2\rangle |\psi_1\rangle) |1\rangle \right],
 \end{aligned} \tag{2.2.8}$$

for which, the probability to measure 0 on the top ancillary qubit is

$$\begin{aligned}
 p(0) &= \frac{1}{4} \left\| |\psi_1\rangle |\psi_2\rangle + |\psi_2\rangle |\psi_1\rangle \right\|^2 = \frac{1}{4} (\langle \psi_1 | \langle \psi_2 |) (|\psi_1\rangle |\psi_2\rangle) \\
 &= \frac{1}{4} (\langle \psi_1 | \langle \psi_2 | \psi_1 \rangle |\psi_2\rangle + \langle \psi_2 | \langle \psi_1 | \psi_2 \rangle |\psi_1\rangle + \langle \psi_1 | \langle \psi_2 | \psi_2 \rangle |\psi_1\rangle + \langle \psi_2 | \langle \psi_1 | \psi_1 \rangle |\psi_2\rangle) \\
 &= \frac{1}{2} \left(1 + \langle \psi_1 | \psi_2 \rangle^2 \right),
 \end{aligned} \tag{2.2.9}$$

assuming $|\psi_1\rangle$ and $|\psi_2\rangle$ real normalized vectors. By sampling the probability, and inverting Equation 2.2.9 we can obtain inner products, and thus calculate bilinear forms. The main difference is that in our circuit we do not encode certain states on given sets of qubits, rather we use the GBEs which also leave some ancillary qubits. The controlled swaps are performed only across the two computational registers, leaving the two ancillary registers untouched. As we will see this change will yield a slightly modified version of Equation 2.2.9. In Figure 2.5b the circuit we propose is drawn. The idea is to split the bilinear form as

$$\vec{y}^T A \vec{x} = \left(\vec{y}^T B \right) (B \vec{x}). \tag{2.2.10}$$

For this reason, given the bilinear form A , we once again consider the element-wise square root matrices $B_{i,j}^\pm \equiv \sqrt{A_{i,j}^\pm}$. We then require $B^\pm$ to be symmetric in order to compute its positive and negative semidefinite parts $B^\pm = B_{>}^\pm - B_{<}^\pm$. If the B matrix is not symmetric it can be symmetrized through

$$B \rightarrow \begin{pmatrix} \mathbf{O} & B \\ B^T & \mathbf{O} \end{pmatrix}. \tag{2.2.11}$$

Since $B_{\geq}^{\pm}$ is always positive semidefinite it admits a real square root, $B_{\geq}^{\pm} = \left(C_{\geq}^{\pm}\right)^2$. We can now show the circuit implementing the swap test. We first block-encode the following states:

$$|C\vec{x}\rangle = \sum_{ik} x_i |i\rangle C_{ki} |k\rangle \quad (2.2.12a)$$

$$|C\vec{y}\rangle = \sum_{jl} y_j |j\rangle C_{jl} |l\rangle \quad (2.2.12b)$$

Thereafter, we perform the swap test over the $|l\rangle, |k\rangle$ elements. The state on the lower registers before the CSWAP is $|\Psi\rangle = |C\vec{x}\rangle |C\vec{y}\rangle$. We call the swapped state $|\bar{\Psi}\rangle$. The swap test returns

$$p(0) = \frac{1}{4} (\langle\Psi| + \langle\bar{\Psi}|) (|\Psi\rangle + |\bar{\Psi}\rangle) = \frac{1}{2} [1 + \langle\Psi|\bar{\Psi}\rangle] \quad (2.2.13)$$

The $|\Psi\rangle$ and $|\bar{\Psi}\rangle$ are

$$|\Psi\rangle = \sum_{ijkl} \frac{\sqrt{|y_i|}}{\|\vec{\psi}_y\|} |i\rangle C_{i,k} |k\rangle \frac{\sqrt{|x_j|}}{\|\vec{\psi}_x\|} |j\rangle C_{l,j} |l\rangle \quad (2.2.14a)$$

$$|\bar{\Psi}\rangle = \sum_{ijkl} \frac{\sqrt{|y_i|}}{\|\vec{\psi}_y\|} |i\rangle C_{l,j} |l\rangle \frac{\sqrt{|x_j|}}{\|\vec{\psi}_x\|} |j\rangle C_{i,k} |k\rangle \quad (2.2.14b)$$

If performing the scalar product $\langle\Psi|\bar{\Psi}\rangle$, we achieve:

$$\sum_{\substack{ijkl \\ \bar{i}\bar{j}\bar{k}\bar{l}}} \frac{\sqrt{|y_i y_{\bar{i}} x_j x_{\bar{j}}|}}{\|\vec{\psi}_y\|^2 \|\vec{\psi}_x\|^2} C_{i,k} C_{l,j} C_{\bar{l},\bar{j}} C_{\bar{i},\bar{k}} \langle i k j l | \bar{i} \bar{l} \bar{j} \bar{k} \rangle \quad (2.2.15)$$

which returns

$$\sum_{ijkl} \frac{|y_i C_{i,k} C_{k,j} x_j C_{l,j} C_{i,l}|}{\|\psi_y\|^2 \|\psi_x\|^2} = \sum_{ij} \frac{|y_i A_{i,k} x_j|}{\|\psi_y\|^2 \|\psi_x\|^2} \quad (2.2.16)$$

since $\sum_k C_{ik} C_{kj} = B_{ij}$ and $B_{i,k}^2 = |A_{i,k}|$. This operation must be repeated for $\vec{y}_{\pm}, \vec{x}_{\pm}, B_{\geq}^{\pm}, A^{\pm}$. In conclusion, by splitting the bilinear form A into two matrices, each applied to both $\vec{x}$ and $\vec{y}$, we can leverage the swap test to calculate the inner product of the two $|\Psi\rangle$ and $|\bar{\Psi}\rangle$, which is ultimately the original result we

intend to calculate, and can be retrieved by Equation 2.2.13. We thus conclude this second Chapter having developed a new set of techniques that, unlike standard BE do not suffer from failing ancillary measurements. With this final Section, we additionally proved that such newly devised techniques also inherit the useful concatenating property from the standard ones. Crucially, it is exactly through this latter approach that in Chapter 3 we find a limit with an exponential speedup with respect to classical computing.

3

USE CASES FOR ALGEBRAIC SPEEDUPS

With the first Chapter we introduced key elements of quantum computing and presented the state of the art with regard to **QRAM** and **BE** techniques. Then, in the second Chapter we devised new quantum algorithms, extending standard **BE** techniques to an always succeeding counterpart. We also showed how such new techniques can be concatenated, allowing for the computation of bilinear forms. In this third and final Chapter we perform a numerical analysis, looking at two different tasks for potential speedups. We start with Section 3.1 in which we carefully address the readout problem for the various techniques, linking the number of shots with the output precision. Having introduced this last key element, in Section 3.2 we proceed with the first task of classification through a single-layer **Neural Network (NN)**. What our numerical analysis concludes is that the simple direct implementation through either the standard or modified **KP** does not provide a quantum speedup. It is for this reason that at the end of the Section we try to delineate some future directions, proposing refinements that empirically reduce the number of operations. We then conclude the Chapter with Section 3.3. Here, we look at the second task, the computation of bilinear forms. What we find is that despite not having a speedup for generic tensors, we do find a remarkable limit. In fact, whenever the tensors have elements with concordant signs, we have an exponential speedup with respect to the classical complexity.

3.1 READOUT OF GENERIC MATRIX VECTOR MULTIPLICATION

In this Section we address the sampling phase of the techniques. In particular, we analytically link a given desired precision ϵ with the number of shots required to achieve that. With Sections 3.1.1 we briefly remind some core statistical scalings,

which are of fundamental use in the analysis. Then, with Section 3.1.2 and 3.1.3 we derive the explicit relations between the number of samples and the standard deviation error of the output.

3.1.1 A brief reminder on statistics

With this Section we report a brief summary of well-known statistical facts. In fact, unlike algorithms like Shor's which encode numbers in the binary representation given by the qubits, we are encoding the numbers in the amplitudes or probabilities of certain states. We thus use the qubit binary representation as just an indexing for the tensors. For this reason it is of high relevance knowing how the precision of the results is dependent on the number of samples we have of such state probabilities. So, given a certain event (like measuring some $|j\rangle$ state) that happens with a probability p_j , the estimated standard error is

$$\hat{\sigma}_{p_j} = \sqrt{\frac{\hat{p}_j (1 - \hat{p}_j)}{S}}, \quad (3.1.1)$$

with $\hat{p}_j$ the estimator of the p_j probability and S the number of samples performed. We then use the reverse of this Equation, meaning we obtain the number of samples we need in order to have a certain precision up to a certain Z -score multiplier [40]

$$S = \frac{Z^2 p (1 - p)}{\epsilon^2}, \quad (3.1.2)$$

where Z is the number of standard deviations and ϵ is the desired precision. Next, if we consider not just one but K mutually exclusive events we still retain Equations 3.1.1 and 3.1.2. We remark how given S shots, despite gaining accuracy on all the p_j at the same time, they are in general associated with different precisions, changing based on the value of p_j . Then, when we consider an expression like Equation 2.1.22

$$\vec{y} = \sum_k \lambda_k \vec{p}^{(k)}, \quad (3.1.3)$$

which is a linear combination of uncorrelated output probabilities from different circuits, the associated error of a given j component of $\vec{y}$ is

$$\hat{\sigma}_{y_j} = \sqrt{\sum_k \frac{\lambda_k^2 \hat{p}_j^{(k)} (1 - \hat{p}_j^{(k)})}{S^{(k)}}}. \quad (3.1.4)$$

We denote with the superscript $\cdot^{(k)}$ the k -th circuit the probability is from, and with the subscript the component, i.e. the specific event probability, of measuring an output $|j\rangle$. We note here how in Equation 3.1.4 we are in general performing a different number of shots $S^{(k)}$ for each circuit. Even though the ideal approach would be to optimize the number of samples depending on the scaling factors and the probabilities, we avoid such an option in favor of performing the same number of shots S for every circuit. The reason is that it yields a much more straightforward analysis, whereas the optimization would need to also take into account a choice on which component to optimize over, cluttering the overall notation and relations. Also, given that the number of circuits is limited (up to 4 or 8 depending on the task) this choice does not constitute a bottleneck as it does not even change the overall complexity class.

3.1.2 Precision scaling of standard BE techniques

We start by looking at standard BE techniques, i.e. KP and FABLE. We report the respective output probabilities as

$$p_j^{(\text{KP})} = \frac{|\sum_{i=0}^{M-1} A_{j,i} x_i|^2}{\|A\|_F^2 \|\vec{x}\|^2} \quad (3.1.5a)$$

$$p_j^{(\text{FABLE})} = \frac{|\sum_{i=0}^{M-1} A_{j,i} x_i|^2}{4^{n+m} \beta^2 \|\vec{x}\|^2} \quad (3.1.5b)$$

In this Section we are going to assume that all output elements $(A\vec{x})_i$ have a well-defined sign, i.e. all positive elements or all negative elements, otherwise we incur into the sign problems we discussed in Section 1.3.3. Then, the value to be computed is the square root of such probabilities, multiplied by a certain scaling

factor. So

$$y_j = \sum_{i=0}^{M-1} A_{j,i} x_i = \|A\|_F \|\vec{x}\| \sqrt{p_j^{(\text{KP})}} \quad (3.1.6a)$$

$$y_j = \sum_{i=0}^{M-1} A_{j,i} x_i = 2^{n+m} \beta \|x\| \sqrt{p_j^{(\text{FABLE})}}. \quad (3.1.6b)$$

For which the standard deviation is given by

$$\hat{\sigma}_{y_j}^{(\text{KP})} = \|A\|_F \|\vec{x}\| \sqrt{\frac{1 - \hat{p}_j}{S}} \quad (3.1.7a)$$

$$\hat{\sigma}_{y_j}^{(\text{FABLE})} = 2^{n+m} \beta \|\vec{x}\| \sqrt{\frac{1 - \hat{p}_j}{S}}. \quad (3.1.7b)$$

Conversely, we require S_j shots in order for the component y_j to reach an ϵ confidence within Z standard deviations

$$S_j^{(\text{KP})} = \|A\|_F^2 \|\vec{x}\|^2 \frac{Z^2 p_j (1 - p_j)}{\epsilon^2} \quad (3.1.8a)$$

$$S_j^{(\text{FABLE})} = 4^{n+m} \beta^2 \|\vec{x}\|^2 \frac{Z^2 p_j (1 - p_j)}{\epsilon^2}. \quad (3.1.8b)$$

3.1.3 Precision scaling of **GBE** techniques

Moving on to the **GBE** techniques, we now have multiple circuits to put together. Here we report the output reconstructions from the various circuit probabilities

$$\begin{aligned} y_j^{\text{KP}} &= \sum_{i=0}^{M-1} \left| A_{j,i}^+ x_i^+ \right| + \left| A_{j,i}^- x_i^- \right| - \left| A_{j,i}^+ x_i^- \right| - \left| A_{j,i}^- x_i^+ \right| \\ &= \|\vec{\psi}_{++}\|^2 p_j^{(++)} + \|\vec{\psi}_{--}\|^2 p_j^{(--)} - \|\vec{\psi}_{+-}\|^2 p_j^{(+-)} - \|\vec{\psi}_{-+}\|^2 p_j^{(-+)} \end{aligned} \quad (3.1.9a)$$

$$\begin{aligned} y_j^{\text{FABLE}} &= \sum_{i=0}^{M-1} \left| A_{j,i}^+ x_i^+ \right| + \left| A_{j,i}^- x_i^- \right| - \left| A_{j,i}^+ x_i^- \right| - \left| A_{j,i}^- x_i^+ \right| \\ &= 2^n \left(\beta_+ p_j^{(++)} + \beta_- p_j^{(--)} - \beta_+ p_j^{(+-)} - \beta_- p_j^{(-+)} \right) \end{aligned} \quad (3.1.9b)$$

with $\|\vec{\psi}_{\pm\pm}\|^2 = \|A^\pm \vec{x}^\pm\|_1$, $\beta_\pm$ the **FABLE** scaling factor for $A^\pm$ as defined in Section 2.1.3, and $\vec{p}^{(\pm\pm)}$ the vector of probabilities associated to the circuits that implements matrix $A^\pm$ (the first sign of $\vec{p}^{(\pm\pm)}$ in the exponent) and $\vec{x}^\pm$ (the second sign of $\vec{p}^{(\pm\pm)}$)

in the exponent). In this second case we have to leverage Equation 3.1.4, and if we assume for simplicity that the same number of samples has been done on each circuit we end up with

$$S_j^{(\text{KP})} = \frac{Z^2}{\epsilon^2} \left(\|\vec{\psi}_{++}\|^4 p_j^{++} (1 - p_j^{++}) + \|\vec{\psi}_{+-}\|^4 p_j^{+-} (1 - p_j^{+-}) \right. \\ \left. + \|\vec{\psi}_{-+}\|^4 p_j^{-+} (1 - p_j^{-+}) + \|\vec{\psi}_{--}\|^4 p_j^{--} (1 - p_j^{--}) \right) \quad (3.1.10a)$$

$$S_j^{(\text{FABLE})} = \frac{Z^2 N^2}{\epsilon^2} \left(\beta_+^2 p_j^{++} (1 - p_j^{++}) + \beta_+^2 p_j^{+-} (1 - p_j^{+-}) \right. \\ \left. + \beta_-^2 p_j^{-+} (1 - p_j^{-+}) + \beta_-^2 p_j^{--} (1 - p_j^{--}) \right). \quad (3.1.10b)$$

3.2 THE CLASSIFICATION TASK

Given the great surge in **Artificial Intelligence (AI)** [41–44] use in recent years, the first use case we consider is the classification task. So, we are provided with a given set of features, for example pixels of an image, and the objective is to assess the class from which those features are extracted. This task can be handled through a (deep) **NN** [45–47], however for two reasons we are going to focus on just a single-layer architecture. The first reason is that this choice is a minimal one. So, if we do not find a speedup it is not due to the complex architecture, but it is an intrinsic problem of the approach. The second reason is that this way the **NN** feedforward (without the activation) is implementable by just a single **BE** application. We in fact have that more complex architectures would not only require more **BE** applications, but also handle the nonlinear activations. As it turns out, the analysis we conduct nonetheless finds that such a simple implementation and architecture still requires too many operations. With Section 3.2.1 we introduce and formalize the classification task. With Section 3.2.2 we show a remarkable property of trained **NN** for which, as we see in the numerical analysis, even standard **BE** is able to be competitive with the **GBE** technique we developed. We then proceed in Section 3.2.4 with a numerical analysis of the classification task. What we deduce from the numerical analysis is that no consistent speedup can be granted by a naive implementation via **BE** techniques. So, in Section 3.2.5 we propose possible future research directions that could overcome such a hurdle.

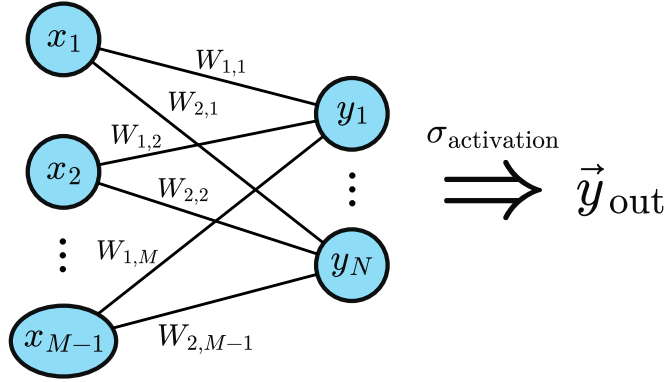


Fig. 3.1 Single-layer **NN**. The $M - 1$ input features enter from the left, which are then multiplied by a weight matrix W producing the $\vec{y} = W\vec{x} + \vec{b}$ vector. The feedforward is then concluded with the application of a certain activation function on the raw $\vec{y}$ output.

3.2.1 The classification task

In a classification task we are provided with a given feature vector $\vec{x}_0 \in D \subset \mathbb{R}^{M-1}$. In the case of images we have that pixels are usually normalized in $D = [0, 1]^{\times(M-1)}$. Then, the single fully connected layer architecture is mathematically described by a weight matrix $W \in \mathbb{R}^{N \times (M-1)}$ and a bias vector $\vec{b} \in \mathbb{R}^N$ as shown in Figure 3.1. Even if it is composed of an input and an output layer, i.e. two layers, we still call it single-layer. The reason is that no computation happens in the input layer, as we just store data in it. The feedforward can be synthesized into a **GEMV** multiplication as follows

$$\vec{y} = W\vec{x}_0 + \vec{b} = (W|\vec{b}) \begin{pmatrix} \vec{x}_0 \\ 1 \end{pmatrix} \equiv A\vec{x}, \quad (3.2.1)$$

with $A \equiv (W|\vec{b})$ and $\vec{x} \equiv \vec{x}_0 \oplus 1$ (with $\oplus$ the direct sum), while the successive activation function is then given by a certain $\sigma_{\text{activation}}$. We remind that the direct sum is defined as

$$\vec{v} \oplus \vec{w} = \begin{pmatrix} v_1 & \dots & v_n & w_1 & \dots & w_m \end{pmatrix} \quad (3.2.2)$$

with $\vec{v} \in \mathbb{R}^n$ and $\vec{w} \in \mathbb{R}^m$. The golden standard for classification is the softmax activation function

$$\sigma_{\text{softmax}}(y_i) \equiv \frac{e^{y_i}}{\sum_{j=0}^{C-1} e^{y_j}}. \quad (3.2.3)$$

A second possible activation function is given by

$$\sigma_{\text{squared}}(y_i) \equiv \frac{|y_i|^2}{\sum_{j=0}^{C-1} |y_j|^2}. \quad (3.2.4)$$

This activation is relevant for standard **BE** techniques, as we encode the result in the amplitudes. When sampling, we therefore have that probabilities are the amplitudes that have undergone exactly the activation function of Equation 3.2.4, i.e. we sample the normalized absolute square of the amplitudes. On the other hand the **GBE** techniques encode the output values in the probabilities, and for this reason we just end up with the output of the **NN** feedforward without any particular activation being natively applied. As we discuss in Section 3.2.3 it both has drawbacks as well as advantages. So, summing up we have that through Equation 3.2.1 we can cast the feedforward multiplication with a single **GEMV** multiplication, and depending on whether we use the standard or modified technique we obtain either an output which undergoes a squared activation function, or a raw feedforward output.

3.2.2 Analysis of the singular values of a trained layers

Having delineated the task, we now show that the failing rate does not disrupt the quantum speedup by itself. In particular, we show that the logical space we introduced in 1.3.5 scales sublinearly with respect to N , and is independent of M . Since we work under the assumption that $N \ll M$, it tells us that such logical space is not shrinking significantly enough to disrupt the advantage. As discussed in Section 1.3.5 we have to study how the ratio of the 2-norm over the Frobenius norm behaves. For such an analysis we then rely on a series of articles by Martin & Mahoney [48–50]. With their work they show how during the training phase the **ESD** distribution of the singular values of the weight matrix evolves from an initial Marchenko-Pastur distribution to a heavy-tailed one when trained, as shown in Figure 3.2. They also found an additional phase where, whenever the **NN** becomes overfitted, there is an additional rank-collapse phase where the singular values tend to vanish. Aside from all the technical definitions, the main takeaway is given by Figure 3.2 (which we report from Ref. [48]) which graphically shows such evolution, from a random-like structure slowly deforming up to reaching a heavy-tailed shape. We now consider the **NN** to be trained and thus we assume it

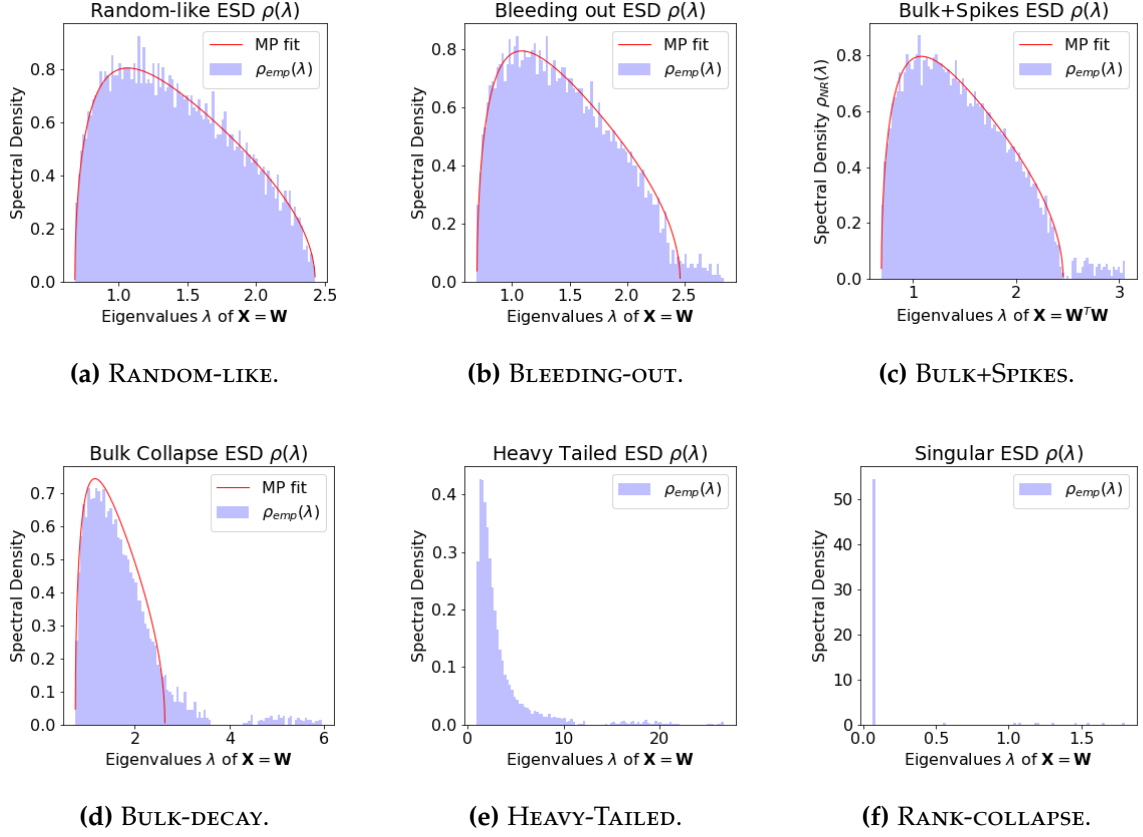


Fig. 3.2 Taxonomy of trained models, plotting the **Empirical Spectral Density (ESD)** of the squared singular values of a weight matrix during the training phase. (a) Starting on the top left panel we have a Marchenko-Pastur distribution, given by a randomly initialized layer. (b) As training starts some singular values start leaking out. (c) Then, the singular values that leaked out separate from the original bulk, forming spikes. (d) As training continues the bulk starts decaying, slowly transitioning into the final phase. (e) Once the layer has been properly trained, the **ESD** assumes a heavy-tailed distribution. (f) If the network is being overtrained and overfitting happens, then the singular values tend to vanish, leading to an extra unwanted phase.

to be in the heavy-tailed case. We can think of the training phase as some initial operation that has to be done just once, and instead shift our interest to all future inferences. So, a heavy-tailed **ESD** is empirically fitted by

$$\rho(\lambda) = \rho_0 \lambda^{-\alpha} \quad \text{with } \lambda \in [\lambda_-, \lambda_+], \quad (3.2.5)$$

with ρ_0 a normalization constant, λ a squared singular value, $\lambda_{\pm}$ the bounds over which this distribution is defined and α the power law exponent. The normalization

ρ_0 is

$$\rho_0 = \frac{1 - \alpha}{\lambda_+^{1-\alpha} - \lambda_-^{1-\alpha}}. \quad (3.2.6)$$

The value of α has a relation to how well trained a layer is. They empirically found that an untrained layer has $\alpha > 4$. As training progresses α ends up roughly in the $[2, 4]$ range, and overfitted layers tend to have $\alpha < 2$. The objective is to associate such an empirical behavior to the ratio of the 2-norm over the Frobenius norm of the weight matrix. We start by taking the **ESD** formula and converting it into a discrete relation for the i -th squared singular value λ_i . It is in fact helpful to derive closed-form analytical results, and it also allows us to go beyond the **ESD** formalism which is not ideal for the matrices that only have a small number N of classes. The way we map the two formalisms is through

$$\frac{i-1}{N-1} = \int_{\lambda_-}^{\tilde{\lambda}_i} \rho(\lambda) d\lambda = \rho_0 \int_{\lambda_-}^{\tilde{\lambda}_i} \lambda^{-\alpha} d\lambda = \frac{\rho_0}{1-\alpha} [\tilde{\lambda}_i^{1-\alpha} - \lambda_-^{1-\alpha}], \quad (3.2.7)$$

where we have $i-1$ in order to keep a 1-based indexing, and $N-1$ in order to have the border condition $\lambda_+ = \lambda_N$. This relation maps the discrete set of ordered singular values $\tilde{\lambda}_i$ to a power law density $\rho(\lambda)$ by equating the normalized rank of the i -th singular value to the cumulative probability distribution function. Thus, integrating $\rho(\lambda)$ gives us an explicit relation that continuously connects the discrete spectra across the bounded domain. We call $\tilde{\lambda}_i$ the singular values that are in ascending order, which we later reversed in descending order into untilded λ_i . By plugging Equation 3.2.6 into Equation 3.2.7, and inverting it for $\tilde{\lambda}_i$ we get the relation

$$\tilde{\lambda}_i = \left[\frac{(i-1)(1-\alpha)}{\rho_0(N-1)} + \lambda_-^{1-\alpha} \right]^{\frac{1}{1-\alpha}} = \left[\frac{(i-1)(\lambda_+^{1-\alpha} - \lambda_-^{1-\alpha})}{(N-1)} + \lambda_-^{1-\alpha} \right]^{\frac{1}{1-\alpha}} \quad (3.2.8)$$

As anticipated we now encode the singular values in descending order, so substituting $j = N + 1 - i$ we get

$$\begin{aligned}
 \lambda_j &= \left[\frac{(N-j)(\lambda_+^{1-\alpha} - \lambda_-^{1-\alpha})}{N-1} + \lambda_-^{1-\alpha} \right]^{\frac{1}{1-\alpha}} = \left[\frac{(N-j)(\lambda_+^{1-\alpha} - \lambda_-^{1-\alpha})}{N-1} + \lambda_-^{1-\alpha} \right]^{\frac{1}{1-\alpha}} \\
 &= \left[N \frac{\lambda_+^{1-\alpha} - \lambda_-^{1-\alpha}}{N-1} + \lambda_-^{1-\alpha} - j \frac{\lambda_+^{1-\alpha} - \lambda_-^{1-\alpha}}{N-1} \right]^{\frac{1}{1-\alpha}} \\
 &= \left(\frac{\lambda_-^{1-\alpha} - \lambda_+^{1-\alpha}}{N-1} \right)^{\frac{1}{1-\alpha}} \left[j - \left(N - \lambda_-^{1-\alpha} \frac{N-1}{\lambda_+^{1-\alpha} - \lambda_-^{1-\alpha}} \right) \right]^{\frac{1}{1-\alpha}}.
 \end{aligned} \tag{3.2.9}$$

We can indeed see how the original power rule of Equation 3.2.5 is mapped into another shifted power rule

$$\lambda_j = K(j - \delta)^{-\beta} \tag{3.2.10}$$

with $\beta \equiv \frac{1}{\alpha-1}$, $K \equiv \left(\frac{\lambda_-^{1-\alpha} - \lambda_+^{1-\alpha}}{N-1} \right)^{\frac{1}{1-\alpha}} = \left(\frac{\lambda_-^{-1/\beta} - \lambda_+^{-1/\beta}}{N-1} \right)^{-\beta}$ and $\delta \equiv \left(N - \lambda_-^{1-\alpha} \frac{N-1}{\lambda_+^{1-\alpha} - \lambda_-^{1-\alpha}} \right) = \left(N - \lambda_-^{-1/\beta} \frac{N-1}{\lambda_+^{-1/\beta} - \lambda_-^{-1/\beta}} \right)$. These operations are well defined since having the exponent $\alpha \in [2, 4]$ (for well-trained layers) leads then to $\lambda_-^{1-\alpha} > \lambda_+^{1-\alpha}$. Among these reparameterizations we highlight the new exponent $\beta \in [1/3, 1]$, with $\beta = 1$ corresponding to $\alpha = 2$ (the overfitting side) which is the most delicate point. The shift parameter δ is restricted to $(-\infty, 1)$ which is good as $\delta \geq 1$ would make the expression diverge or be ill-defined. Additionally, for Figure 3.3 we trained multiple times a certain NN, gathering statistics on the singular values of the trained net. In the left panel we plot their mean with the respective sample standard deviation. In the right panel we instead firstly normalize the singular values so that they sum up to 1, so as

$$\hat{\lambda}_j = \frac{\lambda_j}{\sum_{j=1}^N \lambda_j} \tag{3.2.11}$$

and only then do we perform an average and calculate the standard error of the sample. The remarkable fact is that the error bars get shrunk down significantly in the right panel. This means that during the training the NN is reliably exhibits the same power law behavior up to some normalization, which justifies the variance in the unnormalized panel. Additionally, we note how the tail, i.e. the smaller singular values, decreases faster than the supposed power law. We hypothesize this tendency being due to the fact that the work of Martin & Mahoney were carried

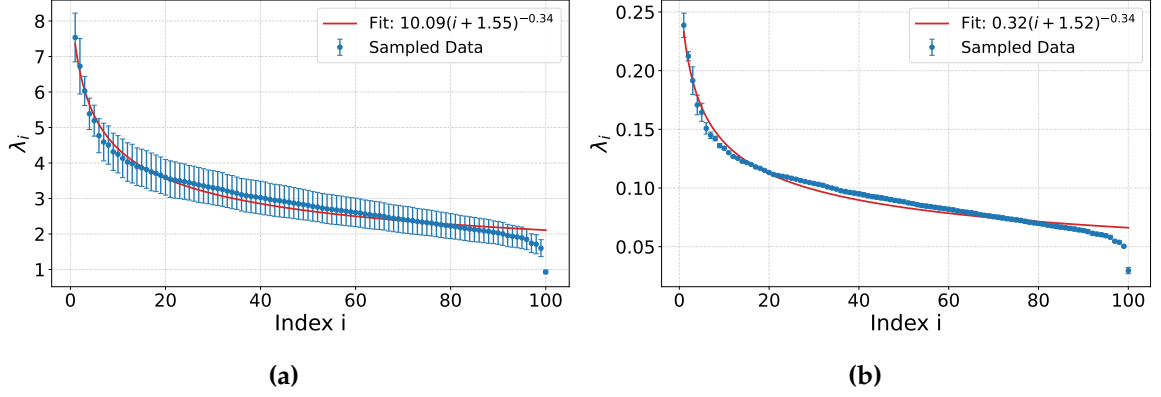


Fig. 3.3 The two panels have been obtained by training 100 times a single-layer network over the CIFAR100 dataset of Table 3.1. In both cases we collect the singular values after the training phase described in Section 3.2.4. (a) In the left panel we plot the average i -th squared singular value, with the associated sample standard deviation. (b) In the right panel, for each run before collecting the squared singular values we normalized them so that they sum up to 1. We then compute again the average and sample standard deviation. As the variance in the right panel is significantly shrunk down, we deduce that the variance of the left panel is mainly due to the **NN** converging to the same power law but with a somewhat free multiplicative factor. Since in the normalized case we have very small errors, it implies that the network stably converges to the same structure.

under the condition of the **NN** being sufficiently large, while we are only working with a simple small **NN**. This non ideality could in fact be the start of a rank collapse, i.e. overfitting, which is reasonable as the architecture is just a single layer. However, as seen in Figure 3.3 the fit is still very good and we defer further discussion to Section 3.2.4. Then, even if we do not explicitly normalize the squared singular values through the **BE** we end up encoding the matrix normalized by its Frobenius norm, so we regardless end up working with these normalized singular values. The 2-norm is

$$\|A\|_2^2 = \lambda_1 = K(1 - \delta)^{-\beta}, \quad (3.2.12)$$

and the Frobenius norm is

$$\|A\|_F^2 = \sum_{j=1}^N \lambda_j = K \sum_{j=1}^N (j - \delta)^{-\beta}. \quad (3.2.13)$$

The normalization K cancels when performing the ratio of the norms, so once again it is independent of such a rescaling. Out of the two we focus on Equation 3.2.13 in order to have closed analytical relationships. As anticipated the delicate point is $\beta = 1$, as for $\beta < 1$ the series diverges for $N \rightarrow \infty$, for $\beta > 1$ the series converges,

while for $\beta = 1$ the series diverges but only logarithmically. We can obtain analytical approximations of Equation 3.2.13 for $\beta \in (0, \infty)$ leveraging the Euler-Maclaurin formula

$$\sum_{j=m}^n f(j) = \int_m^n f(x)dx + \frac{f(n) + f(m)}{2} + \sum_{k=1}^{\lfloor \frac{p}{2} \rfloor} \frac{B_{2k}}{(2k)!} \left(f^{(2k-1)}(n) - f^{(2k-1)}(m) \right) + R_p, \quad (3.2.14)$$

with $\{B_k\}$ the Bernoulli numbers and R_p the error term. So, via Equation 3.2.14 we have that up to a proportionality factor K Equation 3.2.13 becomes

$$\begin{aligned} \sum_{j=1}^N (j - \delta)^{-\beta} &\approx \int_1^N (1 - \delta)^{-\beta} + \frac{(N - \delta)^{-\beta} - (1 - \delta)^{-\beta}}{2} - \frac{\beta}{12} \left((N - \delta)^{-1-\beta} - (1 - \delta)^{-1-\beta} \right) \\ &= \frac{(N - \delta)^{1-\beta} - (1 - \delta)^{1-\beta}}{1 - \beta} + \frac{(N - \delta)^{-\beta} + (1 - \delta)^{-\beta}}{2} - \frac{\beta}{12} \left((N - \delta)^{-1-\beta} - (1 - \delta)^{-1-\beta} \right) \\ &\stackrel{N \gg}{\approx} \frac{(N - \delta)^{1-\beta} - (1 - \delta)^{1-\beta}}{1 - \beta} + \frac{(1 - \delta)^{-\beta}}{2} + \frac{\beta}{12} (1 - \delta)^{-1-\beta}, \end{aligned} \quad (3.2.15)$$

with $B_2 = 1/6$. In order to do further approximations we now need to also assume that $\delta \neq 1$, as when $\delta \rightarrow 1$ we have that the first singular value is significantly spiked with respect to the others. This hypothesis is heuristically consolidated from the numerical examples we discuss in Section 3.2.4. In either case, even if it was not the case having a first spiked singular value makes the ratio of interest close to 1, so it is an optimal limit for the embedding. It is also a theoretically reasonable hypothesis since having most of the information coming from only a singular axis direction means that the information on all the other directions is suppressed, which is usually an unwanted behavior in a **NN**. So, under this assumption we now look at various interesting limits for β . When $\beta < 1$ we just have one diverging term, so

$$\|A\|_F^2 \sim K \frac{N^{1-\beta}}{1-\beta} \quad \text{with } \beta < 1. \quad (3.2.16)$$

Then, $\beta \approx 1$ is the delicate point where, by using the limit $\lim_{x \rightarrow 0} \frac{a^x - 1}{x} = \ln(a)$ we have

$$\|A\|_F^2 \sim K \frac{N^{1-\beta} - (1 - \delta)^{1-\beta}}{1 - \beta} \approx K \ln \left(\frac{N}{1 - \delta} \right) \quad \text{with } \beta \approx 1. \quad (3.2.17)$$

Then, in the limit of $\beta > 1$ the term in $N^{1-\beta}$ vanishes, for which the series is converging independently from N (as long as the previous assumption that N is sufficiently big still holds), for which we have

$$\|A\|_F^2/K \sim -\frac{(1-\delta)^{1-\beta}}{1-\beta} + \frac{(1-\delta)^{-\beta}}{2} + \frac{\beta}{12}(1-\delta)^{-1-\delta} \quad \text{with } \beta > 1. \quad (3.2.18)$$

Now, the last case is when $\beta \gg 1$. By considering few terms of the series we in fact have that as β keeps growing Equation 3.2.18 eventually starts decreasing and converging to 0. However, we know that if the power law is very steep, i.e. $\beta \gg 1$, it means that the major contribution to $\|A\|_F^2$ is given by the first singular value. Thus, in such a limit

$$\|A\|_F^2 \sim \|A\|_2^2 \quad \text{with } \beta \gg 1. \quad (3.2.19)$$

So, we can sum up these notable scaling laws as

$$\|A\|_F^2/K \sim \begin{cases} N^{1-\beta}/(1-\beta) & \beta < 1 \\ \ln\left(\frac{N}{1-\delta}\right) & \beta \approx 1 \\ -\frac{(1-\delta)^{1-\beta}}{1-\beta} + \frac{(1-\delta)^{-\beta}}{2} + \frac{\beta}{12}(1-\delta)^{-1-\delta} & \beta > 1 \\ \|A\|_2^2/K & \beta \gg 1. \end{cases} \quad (3.2.20)$$

So, in the light of this analysis we can go back to the original intent, which was understanding how a standard BE failing rate would behave in such a task. By looking at the ratio of Equation 3.2.12 and 3.2.20 we see that the answer is heavily dependent on the power law parameters, and especially on β . However, given that the ratio of the Frobenius norm over the 2-norm scales sublinearly with N , which we remind is $N \ll M$, it does not constitute a bottleneck. So, even though larger matrices will have more failed runs, this proves that such a fraction of states to be discarded does not increase quickly enough to disrupt the quantum speedup just by itself. So, the result of this Section is that due to the heuristic convergence of the singular values of a trained weight matrix, not only the GBE techniques which are independent from the properties of the A matrix and always succeeding, but even standard BE has the potential to provide a speedup with respect to the classical case. We conclude by briefly noting that even though we encode A and not W , the same analysis on the singular values produces approximately the same results. In

fact as biases $\vec{b}$ tend to converge to relatively small values, the addition of such a new column does not significantly affect the singular values.

3.2.3 *The quantum classifier and its activation functions*

In Section 3.2.1 we delineated the classical workflow of the single fully connected layer. We now present the quantum circuit counterpart. For the sake of simplicity we focus on both the standard and modified KP techniques, but the same can also be extended for the FABLE. For this reason, whenever we talk about the modified counterpart, we always refer to the KP GBE algorithm. As previously discussed, the circuit is composed of just an application of a GEMV multiplication, with the matrix being A , and the vector being $\vec{x} = \vec{x}_0 \oplus 1$. This part can be performed as explained in Sections 1.3.3 and 2.1.2. We now focus on the activation function. In fact, as previously discussed and as shown in Equation 1.3.23 we have that the output of standard BE is not $\vec{y} = A\vec{x}$, but $y_i^2 = (A\vec{x})_i^2$ as we are encoding the results in the amplitudes rather than the probabilities. So, the circuit automatically performs the squared activation function of Equation 3.2.4. The main drawback of the latter is that it breaks the monotonicity of the input, so $y_i > y_j \not\Rightarrow \sigma(y_i) > \sigma(y_j)$. The pivot point of such breaking is the origin, as we are using a parable as an activation function. So, monotonicity is either preserved or reversed as long as the y_i values have the same sign. In any other case it is not possible to reconstruct a correct ordering of the outputs, as we would be able to with a monotone preserving activation as the softmax in Equation 3.2.3. A naive approach could be adding a sufficiently large bias, which despite being feasible it very quickly creates problems in the embedding. By increasing the value of the outputs but keeping the differences the same, due to the normalization of the output we are shrinking the differences we have to resolve in the probabilities. It is for this reason that a more sophisticated alternative is presented in Section 3.2.4. However, a good feature of this activation is that it non linearly makes the top class stand out among all the others. It is in fact from this observation that in Section 3.2.4 we see that how it helps standard BE being competitive with our GBE approach. Next, regarding the KP GBE the discussion is straightforward. In fact, using multiple circuits we are able to extract also information on the sign by encoding the feedforward output directly into the probabilities. It is for this reason that the output can then efficiently undergo any activation function of interest via

classical means. We conclude this Section remarking a very subtle but important detail. Since the monotonicity is not preserved by the standard **BE**, then unless we are able to grant the positivity or negativity of the output y_i , the circuit cannot really be executed as in principle a correct ordering is not assessable. On the other hand, the great advantage the modified approach offers is the robustness against such problems, as it also retrieves information on the sign.

3.2.4 Numerical analysis

In order to conclude the analysis we apply numerical methods to see whether and to what degree a speedup is present through the proposed quantum circuits. Since standard **BE** is not able to retrieve sign information we delay its discussion to the end of this Section. So, we start with our only option for generic classical nets, the **GBE** technique. In Table 3.1 we list the well known and publicly available

Dataset	Shape ($C \times H \times W$)	Total Pixels+1	Classes
BloodCell	$3 \times 240 \times 240$	230,401	4
CIFAR100	$3 \times 32 \times 32$	3,073	100
FashionMNIST	$1 \times 28 \times 28$	785	10
Fruits	$3 \times 100 \times 100$	30,001	257
MNIST	$1 \times 28 \times 28$	785	10
PlantDiseases	$3 \times 256 \times 256$	196,609	38
Rice	$3 \times 250 \times 250$	187,501	5
STL10	$3 \times 96 \times 96$	27,649	10
SVHN	$3 \times 32 \times 32$	3,073	10
Vegetable	$3 \times 224 \times 224$	150,529	15

Table 3.1 Datasets considered in our analysis. They are well known and publicly available datasets devised for classification tasks. All of those datasets are made of images with well defined input size, which we report in the second column. In the third column we explicitly report the number of pixels+1, which corresponds to the number of columns M of A . We then report the number of classes in the last column, which is instead the number of rows N of A .

datasets we considered for the analysis. The common feature they have is that they are all classification datasets where the inputs are images. For this reason the $\vec{x}$ input vector is only positive, and therefore we require just 2 out of 4 possible circuits. As previously justified the architecture is just a single fully connected layer, for minimal testing reasons. During the classical training phase we use

the softmax as the activation function and the Categorical Cross-Entropy for the loss. We choose these as they are standard choices, perfect for characterizing the quantum embedding of typical **NNs**. The training is made of 150 epochs with an early stopping of 10 epochs on the validation loss. However, no model actually reached the 150 epoch limit. A first numerical check we perform is by looking whether the singular values do actually follow the power law of Equation 3.2.10. In Figure 3.3b we plot such an analysis, where we train multiple times a **NN** from random initial conditions, normalize the singular values, extract the statistical mean and error and then fit it with Equation 3.2.10. We measure the accuracy of such a fit via the **Mean Absolute Percentage Error (MAPE)** defined as

$$\text{MAPE}(\{x\}, \{x^{\text{th}}\}) = \frac{100}{n} \sum_{i=1}^n \left| \frac{x_i - x_i^{\text{th}}}{x_i^{\text{th}}} \right|, \quad (3.2.21)$$

with x^{th} the theoretical values (in our case the fitted power law) and x the empirical ones. This indicator tells us on average how far the singular values are with respect to the fitted curve. The **MAPE** values for each dataset are reported in Table 3.2. We perform a first fit with all the singular values, and then a second fit by removing the last 5% of the singular values (or at least the last one for smaller class datasets). From Figure 3.3b and Table 3.2 the trend is overall well respected, but two remarks

	B.C.	C.100	F.M.	F.	M.	P.D.	R.	STL10	SVHN	V.
MAPE	16%	6%	6%	17%	3%	3%	21%	6%	13%	2%
MAPE-95%	3%	4%	5%	13%	3%	2%	6%	1%	1%	2%

Table 3.2 For each dataset we look at the **MAPE** indicator to see the goodness of the power law fit. In the first row we perform this analysis over all the squared singular values. As we noted, the last singular values are vanishing, which makes the **NN** with fewer classes more susceptible to such non ideality. For this reason, by performing an analysis removing the last 5% of singular values (or at least the last one), we can indeed see in the second row how this greatly reduces the **MAPE**.

are due. The first is that we see the end of the tail, i.e. the smaller values, to be badly fitted and tend to vanish too quickly. These last values are the most problematic for the fit, and in fact as we remove them the **MAPE** decreases significantly for the smaller datasets which were highly affected by a last vanishing singular value. Even though its relevance is more evident in datasets with fewer classes, it is actually a common pattern among all datasets, and we hypothesize it is due to overfitting. It

makes in fact sense given the small architecture, which as discussed is more prone to overfitting and thus to beginning the rank-collapse phase where singular values vanish. The second remark is then the fact that in Figure 3.3b the error bars are very small compared to the distance to the heuristic curve. The fact that error bars are small is actually a good sign as it tells us that the NN reliably converges to the same singular value distribution, even though the matrix elements may differ across various trainings. On the other hand, small errors mean that we would fail any compatibility test with the predicted curve. However, we remark how we are not sampling some quantity that should follow a certain physical or mathematical law, rather we are showing that the heuristic law can indeed capture the overall trend. In that regard from Table 3.2 it is clear that such an intent is indeed satisfied.

Next, we move on to the KP GBE technique, and validate how it performs compared to classical means of computation. For each image in the test set we calculate the $\vec{y}^+ = A^+ \vec{x}$ and $\vec{y}^- = A^- \vec{x}$ vectors, with $A^\pm$ from Equation 2.1.21, which is what the quantum circuits of Figure 2.2b return. Then, we calculate the theoretical variance of each $\vec{y} = \vec{y}^+ - \vec{y}^-$ component as in Equation 3.1.4. Next, since we intend to be able to identify the top probability within Z standard deviations we calculate the difference of $y_{\max}$ and all the others y_j with its error

$$\sigma_{y_{\max}-y_j} = \sqrt{\sigma_{y_{\max}}^2 + \sigma_{y_j}^2}. \quad (3.2.22)$$

Concluding, the required number of shots to achieve a Z standard deviations of confidence is the max of those required $S_{\max,j}$ computable via Equation 3.1.1. Performing this analysis on every test sample we can gather some statistics on the typical computational cost. To compare the number of operations required O we have to multiply the depth D of the circuit by the number of shots required S . In Table 3.3 we report the median $\tilde{O}$ and mean $\bar{O}$ number of operations required by the quantum circuit to find the top class, for each dataset, and compare it against the classical cost $O(M)$. The two indicators $\tilde{O}$ and $\bar{O}$ answers two different questions. The median value $\tilde{O}$ is the upper bound of the cost that 50% of the time we have to pay. The second, the mean $\bar{O}$, is instead the average cost we have running the quantum circuit. The median $\tilde{O}$ is always significantly smaller than the mean $\bar{O}$ as in the distribution there are always edge cases that require a very big number of samples that end up skewing the mean metric. We see this kind of behavior from

	B.C.	C.100	F.M.	F.	M.	P.D.	R.	STL10	SVHN	V.
classical	5.4	3.5	2.9	4.5	2.9	5.2	5.3	4.4	3.5	5.2
median	7.6	8.7	5.2	7.9	4.5	8.5	5.3	7.3	7.5	7.9
mean	11.4	11.8	8.7	12.7	7.5	13.5	7.8	10.8	11.3	14.4
depth	2.6	2.6	2.3	2.8	2.3	2.8	2.6	2.6	2.4	2.7

Table 3.3 All entries in this Table are the $\log_{10}$ of the respective value. In the Table we summarize the performance of the **KP GBE** technique. For each dataset we report in the first row the classical parallelized complexity, i.e. the number of pixels+1. Next, we report the mean and median number of quantum operations required, so the product of depth times the number of required samples to assess the top class. Lastly, we also report the depth for each dataset. We note how despite the scaling being polylogarithmic, we are still in most datasets in a range where depth is not negligible.

Figure 3.5a where we plot a histogram of the required number of operations for the Rice test set of Table 3.1, highlighting the median $\tilde{O}$, mean $\bar{O}$ and classical values against such distribution. It is for this reason that the use of both metrics allows for a clearer picture of the typical cost. From Table 3.3 it is clear that no quantum advantage is consistently present. Also, an observation is that although the depth scaling is supposed to be shallow, as it scales like $\mathcal{O}(\log_2(NM)^2)$ which is just a polylogarithmic scaling, for some datasets the depth alone costs similarly to the classical cost. In such cases, even if we were to reduce the number of samples, a relevant speedup would be unfeasible to be achieved. On the other hand, we also have datasets with larger image sizes, which thus lie in a better size regime for which the depth becomes greatly smaller than the classical $\mathcal{O}(M)$ cost. Nonetheless, Table 3.3 clearly shows that we never have a speedup in the total number of operations, so combining the depth with the required number of shots. Only with the Rice dataset we perform competitively, but even there no consistent speedup is achieved.

In this next section we discuss how to be able to perform the **NN** feedforward even with standard **BE**, for which after all we dedicated all Section 3.2.2 about. The core idea is to modify the matrix A , by exploiting the translational invariance of the softmax activation. We in fact have that

$$\sigma_{\text{softmax}}(z_i + c) = \frac{e^{z_i + c}}{\sum_j e^{z_j + c}} = \frac{e^{z_i}}{\sum_j e^{z_j}}. \quad (3.2.23)$$

The idea is therefore that if we push $\vec{y} = A\vec{x}$ to be with a well defined sign, i.e. with positive elements, in principle it does not affect the performance of the network. To achieve such limit we can add a term to the loss that prevents the presence of negative entries in $\vec{y}$

$$\text{loss}'(\vec{y}) = \text{Cross-Entropy} + \lambda \langle e^{\vec{y}^-} - 1 \rangle \quad (3.2.24)$$

with the exponential being performed element-wise, and the mean being over the components. We call this new loss the shifted loss. This modification is just one of many possible. Also, for numerical purposes it is good to put some constraint on the exponential to avoid divergences, especially in the initial steps of the training. Performing a training, or a fine-tuning in such a way, we can reasonably guarantee (and it is also possible to check it with test data) that all output elements are positive. Having granted this basic requirement, it is now possible to use standard **BE** and see the typical cost it has. We report in Table 3.4 the average and median for the standard **BE** of the newly trained **NN**. Once again, for the same reasons it is clear that even this approach is not good enough compared to classical means. However, we like to highlight how this second approach with a modified matrix and using standard **BE** still gives comparable performances with respect to the previous approach reported in Table 3.3. So, as we pointed out in Section 3.2.4 the nonlinear activation that makes the top class stands out, combined with the analysis on the failing rate, makes it so that both techniques end up with competitive performances with each other.

	B.C.	C.100	F.M.	F.	M.	P.D.	R.	STL10	SVHN	V.
classical	5.4	3.5	2.9	4.5	2.9	5.2	5.3	4.4	3.5	5.2
median	7.9	9.7	6.7	9.4	6.6	9.3	7.0	8.3	8.1	8.6
mean	11.5	14.0	10.5	14.2	11.6	13.2	11.9	12.0	13.7	12.3
depth	2.6	2.6	2.3	2.8	2.3	2.8	2.6	2.6	2.4	2.7

Table 3.4 Similarly to Table 3.3 we report the $\log_{10}$ of each entry. In the Table we summarize the standard **KP** performance, having trained the **NN** with the shifted loss of Equation 3.2.24 that grants output values to be all positive. Respectively, the rows are the number of pixels+1, the median $\bar{O}$ and mean $\bar{O}$ number of quantum operations and lastly the depth of the circuit.

3.2.5 *Future research lines to handle the bottlenecks*

In the previous Section we performed an analysis on both the standard and modified **KP** techniques, looking for a speedup. However, the data clearly shows that such speedup cannot be granted relying only on a naive implementation. We start the Section by firstly understanding what is making the required number of samples so big. Then, having understood the problem we propose three possible research lines that have the potential to reduce the number of samples. We remark how the following suggestions are not meant to provide a complete formal analysis. Rather, we want to briefly suggest some ideas and refinements that can be done. We end the Section by showing a relevant example using the Rice dataset from Table 3.1, providing a preliminary numerical support for our claims.

The reason why the required precision for discerning the output probabilities is so high that it disrupts the advantage can be reconducted to two observations. First, performing a classification through a **NN**, which outputs a vector of probabilities whose elements are associated with each class, usually leads to having the top output classes being relatively close. The network is in fact often not too sure on what the class is the true one, and therefore some top classes end up with relatively close values. Secondly, having a lot of classes means that the great majority of them, despite having low probabilities with respect to the top ones, still take up some relevant fraction of the overall probability. In the light of this, we now propose three possible refinements that can improve the cost of the quantum routine.

CIRCUIT PARALLELIZATION We start by observing that the sampling problem only concerns the readout phase, independently from the circuital depth. It is for this reason that in principle we can rely on the key property that sampling is highly parallelizable in nature. In fact, given a larger amount of qubits we can build an arbitrary large amount of circuits which can be executed in parallel as shown in Figure 3.4. This idea is not meant to be an improvement per se, but rather an observation that the overall pipeline can be run in a shorter time if more quantum hardware is available. In addition, unlike classical hardware where devising more powerful GPUs becomes progressively harder, we highlight how this does not hold for the quantum hardware parallelizability counterpart.

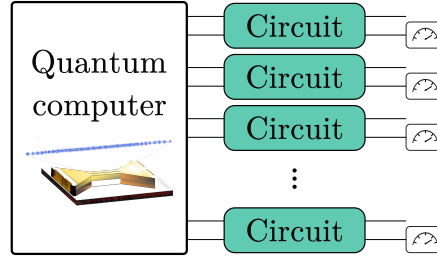


Fig. 3.4 Graphical representation of how with the availability of a larger quantum computer we could allocate more qubits to perform many circuits in parallel. In fact, as the sampling of the output is parallelizable in nature, a larger quantum computer automatically allows for a linear parallelizability of the total number of operations with respect to the number of qubits available. So, even though the overall number of operations is unchanged, the required time to perform such operations is linearly shrunk with the number of circuits.

CLASS FILTERING Next, once again instead of modifying the technique we can change our objective. In fact, if we do not intend to find the top class but rather a given subset of top classes, the required shots to reach the desired precision could be reduced. Then, after finding a given $K \ll N$ subset of top classes we could perform the identification of the top one among those classically, by parallelizing the **GEMV** product of just those K rows. In fact, as long as K is small enough that the whole calculation can be uploaded to the GPU, the cost is then logarithmic rather than linear in M .

MODIFYING THE LOSS Next, the final direction takes inspiration from Equation 3.2.24. It is in fact possible to not just ask that the outputs are positive, but in principle even stronger constraints on the A matrix. For example we could add the constraint that $A^- \approx 0$ by penalizing all negative entries, with the idea of optimizing the **GBE** technique. However, this is a relatively strong requirement that could affect performances. Another modification, which is more typical and present in the literature [51–53] is temperature scaling

$$\vec{y}' = \frac{A\vec{x}}{T}, \quad (3.2.25)$$

where we divide by a given temperature factor T the output before the activation function. If we train the network with a given temperature, what happens is that the outputs converge to the same distribution, meaning that the output $\vec{y} = A\vec{x}$ is similar to the one we get by training instead with a certain temperature $\vec{y}' = A'\vec{x}/T$.

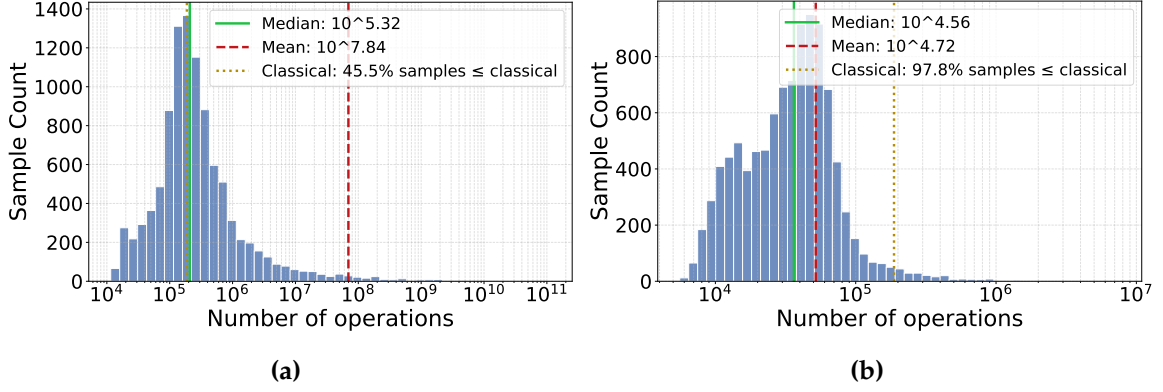


Fig. 3.5 Histogram of the required number of operations distribution using the **KP GBE** algorithm. For both panels we used the test set of the Rice dataset of Table 3.1, performing the analysis on the trained **NN**. In the panels, aside from the histogram distribution we also plot three lines corresponding respectively to the median $\bar{O}$, the mean $\bar{O}$ and the classical cost M . We also highlight in the legend the fraction of samples that are below the classical cost. (a) In the left panel we used the **NN** trained for Table 3.3, so 150 epochs with an early stop of 10 epochs over the validation loss. (b) For the right panel we retrained the **NN** leveraging the insights of Section 3.2.5. In particular the **NN** was trained for 30 epochs with a temperature $T = 100$, and looking for the top two (instead of top one) classes. The interplay of these two modifications is sufficient to provide a relatively consistent improvement. However, repeating this analysis for other datasets shows that it is not sufficient to close the gap, making all of our datasets less costly.

So, by training the **NN** with the temperature scaling of Equation 3.2.25, when we then embed the matrix on the quantum circuit, not involving the T rescaling, we end up with dilated outputs which means that the values are more spread apart and thus more easily distinguishable. In fact, if the training makes A converge so that it outputs same vectors $\vec{y} = A\vec{x} = \vec{y}' = A'\vec{x}/T$, then we have that $\vec{y}' = T\vec{y}$.

We conclude this Section by backing up our claims with a small numerical example. We report in Figure 3.5a the number of operations distribution we have with the **NN** trained in the previous Section for Table 3.3, so using the **KP GBE** approach. On top of the distribution we also highlight the three relevant values of median, mean and classical complexity (number of columns). We then compare this left panel with the right one, Figure 3.5b. In this second panel we train the **NN** with a temperature $T = 100$, and on top of that we do not look for the top class, but rather for the top two classes. The choice of T is constrained only by $T > 1$, as we would otherwise compress the outputs. We chose $T = 100$ as bigger values lead to bad convergences. In that regard, the optimal way would be to gradually increase the temperature rather than start from the beginning with a

too big temperature. The choice of the top two classes is instead justified as the minimal increment from the base, i.e. looking for the top one class. Given that the number of classes of the Rice dataset is already small, even such a small change does affect critically the performance. So, by combining these two refinements we end up with the distribution of Figure 3.5b. So, it is a much better alternative than the direct approach of Figure 3.5a, which also cures the diverging values that skew the mean of the left panel. It is also overall better performing than the classical case. However, by repeating this analysis for also the other datasets, it shows that the refinement helps to reduce only up to very few magnitudes. So, the Rice dataset was the only one to be actually better than the classical case, the reason for which this approach is not sufficient to consistently bridge the gap and make our pipelines better than the classical case. So, we conclude the first classification task with an analysis that shows that despite having some potential for maybe larger image size regimes, the simple application of a BE is not sufficient to provide a speedup for typical NNs if we take into account the whole pipeline, while also showing some potential through the use of more refined techniques.

3.3 QUANTUM SPEEDUP FOR BILINEAR FORM EVALUATIONS

In Section 3.2.1 we looked at the classification task and saw that the application of our techniques, despite being sufficient to perform the computation, is computationally too demanding with respect to the classical counterpart. We identified two core problems consisting of the facts that respectively, a non-negligible portion of the overall probability is taken up by less relevant classes, and the fact that the top classes tend to be close in value makes distinguishing the max probability a demanding task. For this reason we now consider a second task, the computation of a bilinear form $\vec{y}^T A \vec{x}$. Having just a real number as the output does in fact elude both problems, as all the information we extract is not shared across multiple values, and the precision is not forced by the case at hand (high enough to distinguish the maximum value from the second best). As already mentioned in Section 2.2, this task is also relevant in algorithms like SVD or quantum many-body simulations [36–38]. In Section 3.3.1 we are going to briefly relate the number of runs required for a certain precision with the circuit we designed in Section 2.2. Then, with Section 3.3.2 we perform the numerical analysis and end with the conclusions. We find that in the general case, where $\vec{y}$, A and $\vec{x}$ have generic entries,

there is not a speedup as the number of operations quickly exceeds the classical counterpart as the relative error decreases. This said, we also identify a remarkable limit where the number of times the circuit has to be sampled scales only with the relative error, and not the tensor sizes. This happens when all tensors have entries with a well-defined sign. This makes it so that the only scaling on the size is in the depth, which is polylogarithmic. It is thus in such a limit that we do find an exponential speedup in the computation of bilinear forms.

3.3.1 Sampling relations for bilinear form circuits

By reminding the results of Section 2.2.2 we report the probability associated with Equation 2.2.7

$$p_{\pm\pm\pm}(0)^{(\text{KP})} = \sum_{i=0}^{M-1} \sum_{k=0}^{N-1} \frac{y_k^\pm A_{k,i}^\pm x_i^\pm}{\|\vec{\psi}_{\pm\pm}\|^2 y_{\max}^\pm}, \quad (3.3.1)$$

with $\|\vec{\psi}_{\pm\pm}\|^2 = \|A^\pm \vec{x}^\pm\|_1^2$. Here we have up to 8 circuits depending on whether $\vec{y}$, A and $\vec{x}$ are mixed signs tensors or not. By leveraging Equation 3.1.4 we have

$$\begin{aligned} S^{(\text{KP})} &= \frac{Z^2}{\epsilon^2} \sum_{\sigma_1, \sigma_2, \sigma_3 \in \{\pm\}^{\times 3}} \|\vec{\psi}_{\sigma_2, \sigma_3}\|^4 (y_{\max}^{\sigma_1})^2 p_{\sigma_1, \sigma_2, \sigma_3} (1 - p_{\sigma_1, \sigma_2, \sigma_3}) \\ &= \frac{Z^2}{\epsilon^2} \sum_{\sigma_1, \sigma_2, \sigma_3 \in \{\pm\}^{\times 3}} \|A^{\sigma_2} \vec{x}^{\sigma_3}\|_1^2 (y_{\max}^{\sigma_1})^2 p_{\sigma_1, \sigma_2, \sigma_3} (1 - p_{\sigma_1, \sigma_2, \sigma_3}), \end{aligned} \quad (3.3.2)$$

with σ_i the sign component of the i -th tensor. We remind how this Equation is obtained by assuming that we sample the same number of times on all circuits. A remarkable limit is when there is only one circuit. In such a case we would have that the relative error associated with the circuit output

$$r = \vec{y}^T A \vec{x} = p(0) \|A \vec{x}\|_1^2 y_{\max}^2 \quad (3.3.3)$$

is actually independent from the matrix size

$$\frac{\sigma_r}{r} = \frac{\sqrt{\|A \vec{x}\|_1^2 p (1-p) / S}}{p \|A \vec{x}\|_1} y_{\max} = \sqrt{\frac{(1-p)}{Sp}} y_{\max} \quad (3.3.4)$$

by using Equation 3.1.1.

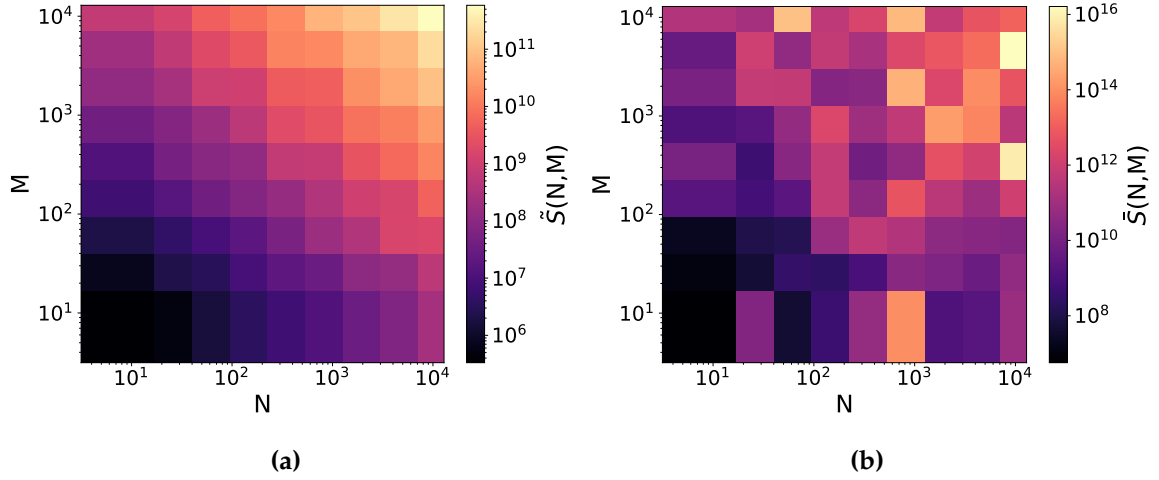


Fig. 3.6 The two panels have been obtained by sampling 100 times the three tensors $\vec{y}$, A and $\vec{x}$ for each N, M matrix shape, with the components extracted from a normal distribution $\mathcal{N}(0, 1)$. In both cases we considered the number of shots $S(N, M)$ required to achieve a relative error of 1%. (a) In the left panel we plot the median value $\tilde{S}$ obtained from the statistical sampling. The values grow as the matrix gets bigger, being slightly less susceptible to N than M . (b) In the right panel we instead plot the mean value $\bar{S}$. The pattern is highly noisy. This is due to the fact that when sampling, the presence of even few divergent values significantly affects the overall mean. We report this second plot for completeness, and to show that the median is a more effective indicator.

3.3.2 Numerical analysis

We now proceed with a numerical analysis, intending to assess the typical performance of the modified approach of Figure 2.3b. We do this analysis by sampling random matrices and vectors. So, by sampling each element from a normal distribution $\mathcal{N}(0, 1)$ centered in 0 with unitary variance. Then, using Equation 3.3.2 we can calculate the required number of samples to reach a 1% precision, i.e. $\sigma_r/r = 0.01$. Through this analysis, which we repeated for various matrix sizes, we obtain Figures 3.6a and 3.6b. In both Figures, but especially in Figure 3.6a, there is a scaling on both N and M even though the scaling tends to be slightly gentler for N . This asymmetry is due to the fact that the $\vec{x} \in \mathbb{R}^M$ is uploaded on the circuit, but the $\vec{y} \in \mathbb{R}^N$ is instead encoded via the Y matrix of Equation 2.2.6. However, the number of required shots itself is far greater compared to the classical counterpart. In fact, classically we would only have a complexity of $\mathcal{O}(\min(N, M))$ assuming we have enough power to parallelize over the smallest dimension, or $\mathcal{O}(NM)$ via unparallelized hardware. In particular what we see in Figure 3.6a is that the number of median samples tends to be $\sim 10^4$ greater than the NM product. On the other

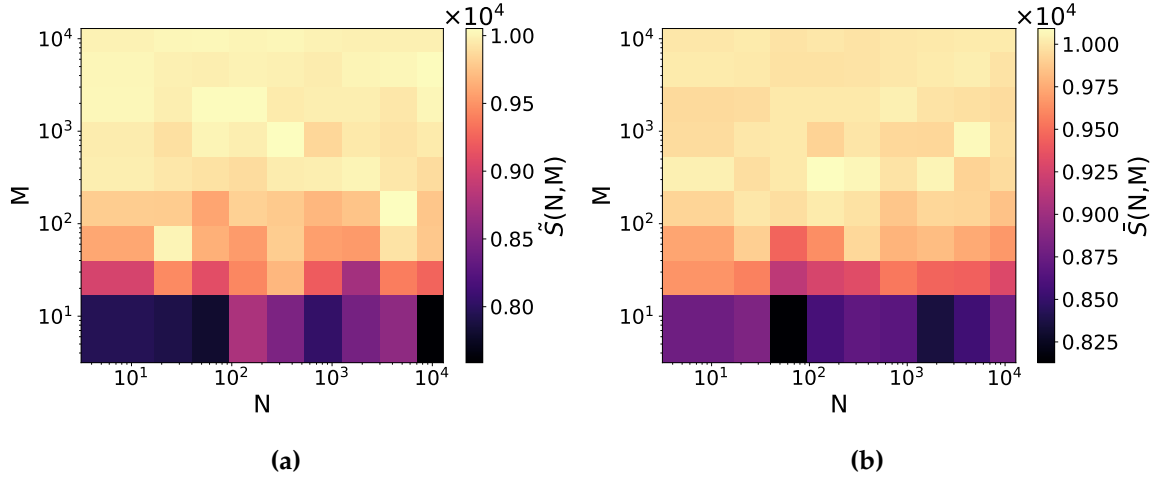


Fig. 3.7 The two panels have been obtained by sampling 100 times the three tensors $\vec{y}$, A and $\vec{x}$ for each N, M matrix shape, with the components extracted from a uniform distribution in $[0, 1]$. In both cases we calculated the number of shots $S(N, M)$ required to achieve a relative error of 1%. (a) In the left panel we plot the median value of shots $\tilde{S}$ obtained from the statistical sampling. We see that the main scaling happens over the M axis, and it is very small. So, the number of samples is fundamentally constant independently from the size. (b) In the right panel we instead plot the mean $\bar{S}$ value of shots. Differently from Figure 3.6b here we do have a pattern also for the mean $\bar{S}$ value. Just as the median $\tilde{S}$ we basically have that the number of required shots is 10^4 , which corresponds exactly to a relative precision of 1% since $S \approx (\sigma_r/r)^{-2} = (0.01)^{-2}$.

hand the mean number of samples of Figure 3.6b is highly noisy. This characteristic is mainly due to the fact that even few highly divergent values can significantly impact the mean. However, even in this second panel we see the values increasing with the size of the matrix. In any case, even if for generic purpose bilinear forms our technique does not provide much advantage, at least in the sampled regime, we do have an interesting limit which we saw in Equation 3.3.4. In fact, the driving factor that disrupts the speedup lies in the fact that the circuits are working against each other. If we instead were in a limit where just one circuit is needed, or more generally when the coefficient factor $\|A^\pm \vec{x}^\pm\|_1^2 (y_{\max}^\pm)^2$ associated with one specific circuit is far greater than the others, then such an elision across circuits would not take place. If we repeat the analysis, but this time we sample the tensor components from a uniform distribution between 0 and 1 in order to guarantee a positive sign, we end up with Figures 3.7a and 3.7b. As pointed out in Equation 3.3.4 the relative error scales basically only over S given that the probability p does not generally tend to converge to 0. So, by looking at both Figure 3.7a and 3.7b it is indeed clear the advantage our technique can provide. In fact not only the median, but also

the mean value lies consistently below 10^4 . This value is not random, in fact as we are sampling up to a 1% relative precision, we have that $S \approx \epsilon^{-2} = (0.01)^{-2} = 10^4$. So, whenever the coefficient associated to a circuit dominates we end in a scenario where rather independently from the tensor sizes the number of shots is constant, which we highlight how remarkable it is. The only size dependence is then left to the circuit depth which, as stated in Section 1.3.3, is $O(\log_2(NM)^2)$, which results exponentially smaller than $O(\min(N, M))$. The number of shots is then only dependent on the relative precision we desire for the computation. We thus conclude our work finding a successful use case where quantum computing, and more in particular our newly developed **KP GBE** algorithm is able to provide an exponential speedup with respect to classical computation. Such a task is the computation of a bilinear form where tensors have (mostly) well defined element signs. We report such results in Table 3.5

	Scaling Law	# of Quantum Circuits
Concordant Signs Bilinear Forms	$O\left(\log_2(NM)^2 \cdot \frac{Z^2 r^2}{\sigma_r^2} \frac{1-p}{p} y_{\max}\right)$	1
Generic Bilinear Forms	$O\left(\log_2(NM)^2 \cdot S^{(KP)}\right)$	2 ~ 8
Classical cost (parallelized)	$O(\max(N, M))$	None

Table 3.5 We report the results obtained in Section 3.3. The concordant sign limit has depth given by Equation 1.3.24, and number of samples by Equation 3.3.4. In such a limit only 1 circuit is necessary. Also, even if we have different signs, as long as one of the circuit coefficients of Equation 3.3.2 dominates we still obtain the same scaling even though we then require multiple circuits. In the second line, whenever we have generic tensors the depth is unvaried. The number of samples is instead strictly larger, given by Equation 3.3.2. Depending on how many tensors have non concordant signs we can have up to 8 circuits. We compare such limits against the classical counterpart, in the third line. Parallelizing over the smaller dimension we have the reported complexity. The scaling of the concordant sign case thus provide an exponential speedup with respect to the classical counterpart.

4

CONCLUSIONS AND PERSPECTIVES

We conclude the work by summing up all the relevant findings and delineating future research lines that could be followed. In Chapter 1 we introduced the **QRAM** and the current literature on **BE** techniques. As discussed at length, in Chapter 1 we highlight how **BE** techniques rely on post-selection measurements to work. This algorithmic choice gives rise to **BE** big shortcoming, failed runs. In fact, whenever the ancillaries are not measured in the 0 state, the quantum state is discarded as useful information cannot be efficiently extracted. It is for this reason that in Chapter 2 we devise a new type of algorithms. Taking inspiration from **BE** techniques and subtly modifying the encoded parametrization and the measured qubits, it is in fact possible to develop an always succeeding counterpart. Our technique is in fact capable of always extracting information from the quantum circuit, regardless of ancillary measurements. Then, always in Chapter 2 we also look at the concatenating property, showing that our **GBE** techniques do inherit such useful capability, but only as long as a proper reparametrization of the tensors takes place.

Having developed a new class of quantum algorithms, in Chapter 3 we look at whether they can provide a speedup, analyzing two relevant tasks. The first is the classification task. So, given an image we leverage a single-layer **NN** architecture to identify the class the image is from. Given that the **NN** feedforward is castable into a single **GEMV** multiplication, the corresponding circuit is made of a single **BE** application. The result we obtain from the analysis is that, for the tested datasets, no speedup is present. On the contrary, the quantum pipeline usually requires significantly more operations than the classical counterpart. Not having found a speedup through a first direct approach, we tried to find ways to decrease the quantum cost. In Chapter 3 we delineate and propose some ideas that can be leveraged to reduce the number of operations required. Hopefully, future research

will combine those suggestions with other techniques, thus achieving an actual speedup.

The second task we considered was the calculation of bilinear forms $\vec{y}^T A \vec{x}$. Through a double application of the **KP GBE** technique it is possible to evaluate such a quantity through a quantum circuit. The numerical analysis we performed found two results. The first result is that for generic tensors $\vec{y}$, A and $\vec{x}$ no speedup is present. However, the second result lies in a remarkable limit. Whenever every component of $\vec{y}$, A , and $\vec{x}$ has a well-defined, independent sign (strictly all positive or all negative), we have that the required number of samples to reach a certain percentual error is independent of the tensor sizes. So, fixing a certain precision, the only dependency on the matrix size is in the depth, which scales polylogarithmically. Since the classical counterpart, leveraging a GPU, scales linearly with one of the two dimensions of A , we thus conclude that in such a limit there is an exponential quantum speedup.



SOFTWARE VERSIONING

In this Appendix we cover the software that has been used during our work. On the analytical side we used Wolfram Mathematica 14.0. For numerical analysis and plotting we used Python. To ensure reproducibility of the calculations all Python dependencies were managed by a virtual environment. In List [A.1](#) we report the used libraries with their version.

```
virtualenv==20.13.0+ds
joblib==1.5.3
kagglehub==1.0.0
kagglesdk==0.1.17
matplotlib==3.10.8
numpy==2.2.6
pennylane==0.42.3
pennylane_lightning==0.42.0
scikit-learn==1.7.2
scipy==1.15.3
seaborn==0.13.2
torch==2.11.0
torchvision==0.26.0
```

Listing A.1 Snippet of requirements.txt content

REFERENCES

- [1] Matteo GA Paris. The modern tools of quantum mechanics: A tutorial on quantum states, measurements, and operations. *The European Physical Journal Special Topics*, 203(1):61–86, 2012.
- [2] Albert Einstein, Boris Podolsky, and Nathan Rosen. Can quantum-mechanical description of physical reality be considered complete? *Physical review*, 47(10):777, 1935.
- [3] John S Bell. On the einstein podolsky rosen paradox. *Physics Physique Fizika*, 1(3):195, 1964.
- [4] Richard P Feynman et al. Simulating physics with computers. *Feynman and computation*, 22:133–153, 2018.
- [5] David Deutsch. Quantum theory, the church–turing principle and the universal quantum computer. *Proceedings of the Royal Society of London. A. Mathematical and Physical Sciences*, 400(1818):97–117, 1985.
- [6] Peter W Shor. Algorithms for quantum computation: discrete logarithms and factoring. In *Proceedings 35th annual symposium on foundations of computer science*, pages 124–134. Ieee, 1994.
- [7] Lov K Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, pages 212–219, 1996.
- [8] David Deutsch and Richard Jozsa. Rapid solution of problems by quantum computation. *Proceedings: Mathematical and Physical Sciences*, pages 553–558, 1992.
- [9] András Gilyén, Yuan Su, Guang Hao Low, and Nathan Wiebe. Quantum singular value transformation and beyond: exponential improvements for quantum matrix arithmetics. In *Proceedings of the 51st annual ACM SIGACT symposium on theory of computing*, pages 193–204, 2019.
- [10] John M Martyn, Zane M Rossi, Andrew K Tan, and Isaac L Chuang. Grand unification of quantum algorithms. *PRX quantum*, 2(4):040203, 2021.
- [11] A Yu Kitaev. Quantum measurements and the abelian stabilizer problem. *arXiv preprint quant-ph/9511026*, 1995.
- [12] Aram W Harrow, Avinatan Hassidim, and Seth Lloyd. Quantum algorithm for linear systems of equations. *Physical review letters*, 103(15):150502, 2009.
- [13] Seth Lloyd. Universal quantum simulators. *science*, 273(5278):1073–1078, 1996.

- [14] Scott Aaronson. Read the fine print. *Nature Physics*, 11(4):291–293, 2015.
- [15] Samuel Jaques and Arthur G Rattew. Qram: A survey and critique. *Quantum*, 9:1922, 2025.
- [16] Koustubh Phalak, Avimita Chatterjee, and Swaroop Ghosh. Quantum random access memory for dummies. *Sensors*, 23(17):7462, 2023.
- [17] Mikko Mottonen, Juha J Vartiainen, Ville Bergholm, and Martti M Salomaa. Transformation of quantum states using uniformly controlled rotations. *arXiv preprint quant-ph/0407010*, 2004.
- [18] Vittorio Giovannetti, Seth Lloyd, and Lorenzo Maccone. Quantum random access memory. *Physical review letters*, 100(16):160501, 2008.
- [19] Srinivasan Arunachalam, Vlad Gheorghiu, Tomas Jochym-O’Connor, Michele Mosca, and Priyaa Varshinee Srinivasan. On the robustness of bucket brigade quantum ram. *New Journal of Physics*, 17(12):123010, 2015.
- [20] Fanhao Shen, Yujie Ji, Debin Xiang, Yanzhe Wang, Ke Wang, Chuanyu Zhang, Aosai Zhang, Yiren Zou, Yu Gao, Zhengyi Cui, et al. A bucket-brigade quantum random access memory. *Nature Physics*, 22(5):745–750, 2026.
- [21] Olivia Di Matteo, Vlad Gheorghiu, and Michele Mosca. Fault-tolerant resource estimation of quantum random-access memories. *IEEE Transactions on Quantum Engineering*, 1:1–13, 2020.
- [22] Iordanis Kerenidis and Anupam Prakash. Quantum recommendation systems. *arXiv preprint arXiv:1603.08675*, 2016.
- [23] Andrew M Childs and Nathan Wiebe. Hamiltonian simulation using linear combinations of unitary operations. *arXiv preprint arXiv:1202.5822*, 2012.
- [24] Dominic W Berry, Andrew M Childs, Richard Cleve, Robin Kothari, and Rolando D Somma. Simulating hamiltonian dynamics with a truncated taylor series. *Physical review letters*, 114(9):090502, 2015.
- [25] Guang Hao Low and Isaac L Chuang. Hamiltonian simulation by qubitization. *Quantum*, 3:163, 2019.
- [26] Daan Camps and Roel Van Beeumen. Fable: Fast approximate quantum circuits for block-encodings. In *2022 IEEE International Conference on Quantum Computing and Engineering (QCE)*, pages 104–113. IEEE, 2022.
- [27] Parker Kuklinski and Benjamin Rempfer. S-fable and ls-fable: Fast approximate block-encoding algorithms for unstructured sparse matrices. *arXiv preprint arXiv:2401.04234*, 2024.
- [28] Daan Camps, Lin Lin, Roel Van Beeumen, and Chao Yang. Explicit quantum circuits for block encodings of certain sparse matrices. *SIAM Journal on Matrix Analysis and Applications*, 45(1):801–827, 2024.

- [29] Christoph Sünderhauf, Earl Campbell, and Joan Camps. Block-encoding structured matrices for data input in quantum computing. *Quantum*, 8:1226, 2024.
- [30] Zexian Li, Xiao-Ming Zhang, Chunlin Yang, and Guofeng Zhang. Binary tree block encoding of classical matrix. *IEEE Transactions on Quantum Engineering*, 2025.
- [31] Quynh T Nguyen, Bobak T Kiani, and Seth Lloyd. Block-encoding dense and full-rank kernels using hierarchical matrices: applications in quantum numerical linear algebra. *Quantum*, 6:876, 2022.
- [32] Shantanav Chakraborty, András Gilyén, and Stacey Jeffery. The power of block-encoded matrix powers: improved regression techniques via faster hamiltonian simulation. *arXiv preprint arXiv:1804.01973*, 2018.
- [33] Hao Shi, Guofeng Zhang, and Ming Zhang. A quantum algorithm for the kalman filter using block encoding. *arXiv preprint arXiv:2404.04554*, 2024.
- [34] Fino and Algazi. Unified matrix treatment of the fast walsh-hadamard transform. *IEEE Transactions on Computers*, 100(11):1142–1146, 1976.
- [35] Sebastiano Corli, Giorgio Panichi, Samuele Altilia, Edoardo Suerra, Simone Cialdi, and Enrico Prati. Physical implementation of analytic derivatives on a photonic quantum computer. *Physical Review Applied*, 26(1):014107, 2026.
- [36] Maria Schuld and Nathan Killoran. Quantum machine learning in feature hilbert spaces. *Physical review letters*, 122(4):040504, 2019.
- [37] Massimiliano Incudini, Francesco Martini, and Alessandra Di Pierro. Toward useful quantum kernels. *Advanced Quantum Technologies*, 8(12):2300298, 2025.
- [38] Adam Smith, Myungshik S Kim, Frank Pollmann, and Johannes Knolle. Simulating quantum many-body dynamics on a current digital quantum computer. *npj Quantum Information*, 5(1):106, 2019.
- [39] Harry Buhrman, Richard Cleve, John Watrous, and Ronald De Wolf. Quantum fingerprinting. *Physical review letters*, 87(16):167902, 2001.
- [40] Wiliam G Cochran. Sampling techniques 3rd edjohn wiley and sons. *New York, NY*, 428, 1977.
- [41] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [42] Rishi Bommasani, Drew A Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, et al. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*, 2021.

- [43] George Em Karniadakis, Ioannis G Kevrekidis, Lu Lu, Paris Perdikaris, Sifan Wang, and Liu Yang. Physics-informed machine learning. *Nature Reviews Physics*, 3(6):422–440, 2021.
- [44] Giuseppe Carleo and Matthias Troyer. Solving the quantum many-body problem with artificial neural networks. *Science*, 355(6325):602–606, 2017.
- [45] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. *nature*, 323(6088):533–536, 1986.
- [46] George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of control, signals and systems*, 2(4):303–314, 1989.
- [47] Frank Rosenblatt. The perceptron: a probabilistic model for information storage and organization in the brain. *Psychological review*, 65(6):386, 1958.
- [48] Michael Mahoney and Charles Martin. Traditional and heavy tailed self regularization in neural network models. In *International Conference on Machine Learning*, pages 4284–4293. PMLR, 2019.
- [49] Charles H Martin and Michael W Mahoney. Heavy-tailed universality predicts trends in test accuracies for very large pre-trained deep neural networks. In *Proceedings of the 2020 SIAM International Conference on Data Mining*, pages 505–513. SIAM, 2020.
- [50] Charles H Martin and Michael W Mahoney. Implicit self-regularization in deep neural networks: Evidence from random matrix theory and implications for learning. *Journal of Machine Learning Research*, 22(165):1–73, 2021.
- [51] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.
- [52] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q Weinberger. On calibration of modern neural networks. In *International conference on machine learning*, pages 1321–1330. PMLR, 2017.
- [53] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *International conference on machine learning*, pages 1597–1607. PmLR, 2020.